SaraswathAI : An Agentic Blockchain-Integrated Domain-Specific LLM Platform for Optimized Educational Content Retrieval, Personalized Learning, and Intelligent Assessment.

MAJOR PROJECT REPORT submitted in partial fulfillment of the requirements

for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Neeraja Gurram 228W1A12F5

Lokesh Narne 228W1A12H4

Under the Guidance of

Dr. M. Ramesh,

Senior Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

V R SIDDHARTHA ENGINEERING COLLEGE

(AUTONOMOUS - AFFILIATED TO JNTU-K, KAKINADA)

Approved by AICTE &Accredited by NBA

KANURU, VIJAYAWADA-520007

ACADEMIC YEAR

(2025-26)

V.R.SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous)
(An ISO certified and NBA accredited institution)
Kanuru, Vijayawada – 520007

CERTIFICATE

This is to certify that this project report titled "**SaraswathAI : An Agentic Blockchain-Integrated Domain-Specific LLM Platform for Optimized Educational Content Retrieval, Personalized Learning, and Intelligent Assessment**." is a bonafide record of work done Neeraja Gurram (228W1A12F5), Lokesh Narne (228W1A12H4) under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V.R. Siddhartha Engineering College** (Autonomous under JNTUK) during the year 2025-26.

Dr.M.Ramesh	**(Dr. M. Suneetha)**
Senior Assistant Professor	Dean R&D, Professor &Head
Dept. of Information Technology	Dept. of Information Technology

EXTERNAL EXAMINER SIGNATURE

Date of examination:

DEPARTMENT OF INFORMATION TECHNOLOGY

V.R.SIDDHARTHA ENGINEERING COLLEGE

PROJECT SUMMARY

S.No	Item	Description
1	**Project Title**	SaraswathAI : An Agentic Blockchain-Integrated Domain-Specific LLM Platform for Optimized Educational Content Retrieval, Personalized Learning, and Intelligent Assessment
2	**Student Names & Numbers**	Neeraja Gurram (228W1A12F5) Lokesh Narne (228W1A12H4)
3	**Name of The Guide**	Dr.M.Ramesh, Senior Assistant Professor.
4	**Research Group**	AI, ML, DL.
5	**Application Area**	Education
6	**Aim of the Project**	To develop an intelligent agentic learning system that integrates content retrieval, adaptive assessment, and personalized feedback for enhanced educational outcomes, ensuring security, transparency, and structured learning through the use of knowledge graphs, adaptive algorithms, and blockchain-based verification.
7	**Project Outcomes**	1. Students will receive continuous learning support through automated practice and performance evaluation after content interaction. 2. The system will enable adaptive learning experiences by identifying knowledge gaps and guiding personalized improvement. 3. Learning effectiveness and knowledge retention will improve through structured assessment and feedback mechanisms. 4. The platform will evolve from a passive information provider into an intelligent and self-guided learning environment.

Student Signatures

1.

2.

3.

Signature of the Guide

ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **V.R SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our Principal **Dr. A.V.RATNA PRASAD**, for his encouragement and support all through the way of my project.

On the submission of this Project report, I would like to extend my honour to **Dr. M.Suneetha**, Dean R&D, Professor & Head of the Department, IT for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr. M. Ramesh, Senior Assistant Professor** for his guidance and assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

TABLE OF CONTENTS

ACKNOWLEDGEMENT iii

LIST OF FIGURES v

LIST OF TABLES vi

ABSTRACT viii

CHAPTER-1 Introduction 9

1.1 Origin of the Problem 9

1.2 Basic definitions and Background 10

1.3 Problem Statement with Objectives and Outcomes 11

1.4 Related Applications 12

CHAPTER-2 Review of Literature 14

2.1 Description of Existing Systems 14

2.2 Summary of Literature Study 19

CHAPTER-3 Proposed Method 22

3.1 Design Methodology 22

3.2 System Architecture Diagram 25

3.3 Description of Algorithms 32

3.4 Description of Datasets, Requirements and Tools 43

CHAPTER-4 Results and Observations 46

4.1 Stepwise description of Results 46

4.2 Result Analysis 51

4.3 Observations from the work 59

CHAPTER-5 Conclusion and Future work 46

5.1 Conclusion 46

5.2 Future work 51

REFERENCES

Appendix

LIST OF FIGURES

Figure **Page**

Figure 3.1: Use Case Diagram 26

Figure 3.2: Architecture Diagram 27

Figure 3.3: TF-IDF Mathematical Formula 29

Figure 4.1: Main Dashboard – Learning Mode Selection Interface 52

Figure 4.2: Knowledge Graph – Topic Visualization After Document Processing 53

Figure 4.3: Learning Path – Topologically Sorted Study Sequence 54

Figure 4.4: Study Mode – AI-Generated Explanation for "User Authentication" 55

Figure 4.5: Study Mode – Code Example Generated for "User Authentication 56

Figure 4.6: Quiz Mode – Adaptive Multiple-Choice Question Interface 57

Figure 4.7: Topic Type Distribution Chart 58

Figure 4.8: Knowledge Graph After Session 59

Figure 4.9: Mini Project II – System initialization with blockchain verification activated alongside GPU-accelerated RAG 61

Figure 4.10: Mini Project I – Query response time 64

Figure 4.11: Mini Project II – Blockchain test status report 65

Figure 4.12: Mini Project I – Chat-based home interface 65

LIST OF TABLES

Table Name	Page
Table 3.1: Software Requirements	45
Table 3.2: Hardware Requirements	46
Table 4.1: Summary of Comparative Analysis	18

ABSTRACT

More students are using AI-enabled education platforms to make finding academic materials easier than ever before, but most of these systems passively provide students with resources but do not provide features like organized learning plans, continuous assessment, or personalized feedback. This article presents SaraswathAI, a specialized language model platform that converts document-based educational resources into a guiding and adaptive educational process. This system uses tools such as NetworkX and Betweenness Centrality to create a Knowledge Graph by analyzing the topics and how they are related to each other and their importance, and then organizes the topics logically using a Topological Sort, based on the sequential order of any prerequisite topics. The system utilizes a Retrieval-Augmented Generation module that utilizes the FAISS vector database and local applications of language models to provide students with the appropriate content for their individual studies based on their selected topic. The assessment component of this system provides adaptive difficulty by adjusting the level of difficulty of the assessments as students' progress through the assessments, using Mastery-Based Progression and Adaptive Difficulty methods. This platform builds on the previous development of the fastest methods for retrieving data using GPUs, and developing a method for confirming the integrity of the data (i.e. through the use of Blockchain services) by developing a method for autonomously handling the entire learning process, rather than simply waiting for the learner to provide input. Testing on curriculum-based documents demonstrates that SaraswathAI can create structured topic maps and also can develop suitable study plans by considering the dependencies between the various topics.

Keywords: *Agentic AI, Blockchain Integration, Personalized Learning, Knowledge Graphs, Large Language Models (LLMs), Adaptive Learning, Intelligent Assessment, Educational Technology.*

Chapter -1 Introduction

1.1 Origin of the Problem

The emergence of digital technologies has led to a dramatic transformation in the education industry and has enabled users to access academic material from the internet. Systems such as e-learning, intelligent tutoring systems, and educational applications powered by artificial intelligence have facilitated the learning process through scalability, flexibility, and personalization. However, despite the progress made by the technology in the sector, most platforms still offer only the delivery of content and the acquisition of knowledge without adequate mechanisms to facilitate continuous assessment and mentorship in the educational process.

Presently, learners utilize platforms for the purposes of searching, browsing, or attending to academic courses and lectures without having means to assess their progress or practice acquired skills and knowledge. Consequently, learners end up engaging in the consumption of huge amounts of data without necessarily assimilating the information. Besides, platforms do not offer a means of dynamic adaptation in order to personalize the learning process according to each individual's unique capabilities.

Another major issue is the lack of learning paths that are based on intelligent algorithms. Although the abundance of educational materials has always been a major advantage of traditional platforms, there has never been enough information on how to move through subjects in an optimized way. It has become increasingly difficult for learners to understand the order in which knowledge should be acquired or which aspects of the subject are particularly significant. The issue that also becomes especially noticeable is the lack of tools that would allow monitoring the progress of one's learning.

In the modern world where automation and artificial intelligence are becoming increasingly important components of educational activities, traditional approaches become even less relevant. Contemporary students require platforms that are able to interact with them, evaluate their skills, and provide adequate feedback in a timely manner. Another pressing issue in terms of modern technologies is related to their ability to work effectively in an environment characterized by issues of data security and authenticity.

All these constraints together define the source of the problem tackled in this study. It is evident that there is a strong need for an intelligent, integrated solution that can not only access appropriate learning material but can also ensure ongoing assessments and facilitate adaptive and personalized learning. The objective of this study is to develop such a platform using artificial intelligence and blockchain technology.

1.2 Basic Definitions and Background

The evolution of intelligent learning systems is made possible through progress in various fields including but not limited to Artificial Intelligence (AI), Machine Learning (ML), Natural Language Processing (NLP), and distributed systems. The knowledge of underlying concepts in these technologies is required to fully grasp the workings of the suggested system.

Artificial Intelligence (AI) can be defined as intelligence demonstrated by machines which allow them to reason, learn, and solve problems similar to human beings. From an educational point of view, AI plays an important role in building systems able to observe student behavior, offer customized content, and assess them.

Large Language Models (LLMs) are a type of AI model trained using huge volumes of textual data in order to generate language similar to that used by humans. Large Language Models are the base for building intelligent educational systems able to search for information and ask questions, among other capabilities

Agentic systems provide a novel approach to artificial intelligence wherein machines function autonomously and perform various activities, make decisions, and adjust their behavior according to the results obtained. Agentic systems may control the whole process of studying, including delivering educational materials, evaluating the learners' progress, and providing feedback, without the need for constant human involvement

Knowledge graphs are graphical models of knowledge where different notions are represented as vertices, and relations between them are shown as edges. Knowledge graphs are utilized in education to structure educational materials logically. The purpose is to establish connections between topics and create the optimal order for studying them.

Adaptive Learning entails learning systems whose curriculum changes and adjusts itself according to the level and performance of the learners. It is aimed at improving user experience and ensuring that each learner receives customized learning.

On the other hand, Blockchain Technology is a decentralized system used to record transactions in a tamper-resistant ledger. Within the scope of this project, blockchain technology is important as it helps ensure that educational data are authentic, transparent, and trustworthy.

Apart from the above-mentioned key elements, the current generation of educational platforms also requires embeddings models and vector databases to make it possible to store and retrieve learning materials semantically. Some popular tools used for this purpose include LangChain, FAISS/ChromaDB, and visualization libraries.

Therefore, combining all the above-discussed technologies will be critical in creating next-generation educational platforms. Specifically, incorporating these tools into educational applications will help develop intelligent learning platforms rather than just content repositories

1.3 Problem Statement, Objectives and Outcomes

Problem Statement

While there may be greater access to educational materials, the current methods used by existing platforms fail to address learning in an active way. This is because such platforms do not track the student's comprehension and progress. This calls for the development of systems capable of tracking learning.

Objectives

- To develop the SaraswathAI platform to incorporate an agentic approach that can independently oversee the process of learning without being confined to queries.
- To develop techniques for generating quizzes automatically from curriculum-based educational data retrieved from the database.
- To develop intelligent analysis tools for evaluating students' answers and giving relevant academic feedback.

- To combine assessment-based learning with the current secure and customized educational data retrieval system.

Outcomes

- Continuous support for students' learning process through automated practice and performance evaluation after content consumption will be provided.
- The tool will offer adaptive learning experience, as it will help identify knowledge gaps and assist in personal development.
- Efficiency of learning and knowledge acquisition will increase due to the implemented testing and feedback system.
- The tool will progress from a passive information source into an active learning environment.

1.4 Related Applications

The proposed SaraswathAI platform, which integrates agentic AI, adaptive learning mechanisms, and blockchain technology, addresses critical needs across multiple domains where intelligent, personalized, and secure learning is essential.

Educational Applications

The role of the system within the framework of formal education is to serve as an intelligent tutor that not only facilitates the delivery of content but also provides explanations based on the curriculum, automated testing, and intelligent feedback. In contrast to other systems that merely act as sources of information retrieval, the SaraswathAI system guarantees that learners interact with educational content by means of ongoing testing and assessment. The system facilitates the preparation for exams by offering quizzes, assessing the learner’s weaknesses, and practicing learned material.

Institutional Applications

EduTech organizations will benefit from this platform by optimizing learning supervision, management of resources, and information transparency. The platform facilitates the monitoring of student progress, their performance, and interactions.

Educators are provided with an opportunity to make evidence-based decisions. Academic staff can store and control resources such as learning content, assignments, and references in an organized manner. By adopting blockchain technology, EduTech institutions will ensure that all academic activities, assessments, and performance are stored securely and are transparent.

Industrial and Corporate Applications

Within organizations, SaraswathAI can serve as a valuable tool in training and managing knowledge within the organization. The platform can be used to conduct training sessions, evaluate the effectiveness of training using automated assessments, and recommend appropriate training modules for individuals based on their proficiency levels. The platform will ensure that only authentic information is made available to the individuals while maintaining detailed records of all training activities undertaken. For sectors that have stringent requirements for compliance and validation of documents, the use of blockchain technology will be useful in facilitating these objectives.

Chapter -2 Review of Literature

The evolution of intelligent educational systems has greatly benefited from the innovations within the realms of agentic artificial intelligence, subject-oriented large language models, and adaptive learning systems. The recent advancements within this field have focused on the shift from passive information retrieval systems towards active and evolving AI-powered educational ecosystems. This chapter provides a detailed review of the existing literature, which focuses on such important areas as self-evolving AI agents, LLM-based educational retrieval systems, engagement frameworks, personalized learning paths, and agentic workflows within the education space.

2.1 Description of Existing Systems

2.1.1 A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems [1]

Year: 2025

Dataset(s): Survey-based study covering diverse benchmarks across biomedical, programming, and finance domains

The survey offers a comprehensive overview of self-evolving AI agents which are those that independently improve themselves by continuously interacting with the data and the environment in which they function. The paper proposes a conceptual agentic framework which is comprised of four elements, namely, System Inputs, Agent Systems, Environment, and Optimizers. In particular, the paper explores ways in which language models can utilize feedback loops and environmental stimuli in conjunction with memory to improve their own performance without undergoing retraining. Besides this, the paper considers various approaches to evaluating the performance of such agents as well as the evolution strategies applied within specific domains.

Observations:

The survey sets the groundwork for theoretical study of the agentic AI paradigm in relation to autonomous task management and continuous self-improvement. The unified approach adopted by this paper serves as an effective theoretical template for the development of SaraswathAI as an agentic system, especially in terms of its

implementation of sequential workflows including document analysis, knowledge graph creation, content delivery, evaluation, and mastery tracking. Nevertheless, the survey does not cover the requirements of agents that evolve within a more rigidly structured environment like an educational setting, including curriculum adherence, modelling knowledge dependencies, and mastery-based progression. Indeed, an important point that can be noted in this regard is that while task completion is essential in any agentic system, education-oriented agents should ideally be able to demonstrate a clear improvement in the level of understanding of learners.

2.1.2 LLM-Based Educational Content Retrieval and Intelligent Query Processing [2]

Year: 2026

Dataset(s): Curriculum-specific educational document corpora and benchmark information retrieval datasets

This paper focuses on the use of large language models for retrieving educational content from specific domains. This paper evaluates optimal retrieval methods by employing the RAG pipeline and neural indexing techniques, assessing their performance in terms of enhancing the comprehension of the queries and generating accurate responses in knowledge-rich academic environments. The authors provide improvements to the retrieval pipeline by optimizing document segmentation, aligning embeddings, and prompt engineering based on the domain. Evaluations on different curriculum datasets have shown that retrieval methods driven by LLMs perform better than keyword-driven methods in terms of accuracy and relevance. There are still challenges in addressing student queries with ambiguity and insufficient specifications.

Observations:

Indeed, this piece of research makes a valuable addition to solving the issue of content retrieval accuracy based on curriculum relevance, and its conclusions strongly support the choice of the RAG architecture implemented in SaraswathAI. The proof that domain-specific optimisation of retrieval processes positively influences response quality justifies the decision to focus only on student-generated academic papers, instead of using more general web sources. On the other hand, the described system still stays limited in functionality, since there is no measure taken into account to

evaluate whether the student has understood the material and received adequate help. Moreover, there is no opportunity provided for creating individual learning paths based on this material and further evaluation of the student's progress. All these issues have been successfully solved by SaraswathAI.

2.1.3 AI-Driven Learning Interaction and Engagement Framework for Digital Education [3]

Year: 2026

Dataset(s): Student interaction logs and digital learning platform usage data

The proposed research introduces an innovative AI-powered approach to analysing and enhancing student engagement in digital learning platforms. By constantly tracking engagement metrics such as session length, search volume, navigation patterns, and response time, the model can predict signs of disengagement before they materialise into dropping out and underperformance. In cases of detected disengagement, the model will implement adaptive strategies such as reorganising content, changing difficulty levels, and encouraging motivation. The study also examines the possibility of embedding AI agents in current learning management platforms in order to conduct continuous and seamless performance monitoring. According to the findings of this research, intelligent and data-fuelled feedback mechanisms can have a profound positive effect on student retention and engagement.

Observations:

Firstly, the issue identified in this framework is real and crucial since students disengage from the learning platform without triggering any responses from the system. The relevance of the framework in relation to the philosophy of SaraswathAI lies in its ability to show how continuous monitoring and intervention could enhance the retention rate among students. Nevertheless, the interventions offered in the framework appear to be responsive rather than proactively building an engaging system architecture. This is critical since SaraswathAI offers structured learning progression and evaluation rather than unstructured learning where students simply browse the information presented by the platform. Secondly, unlike SaraswathAI, the framework fails to include a knowledge graph that would enable learners to progress in their learning based on mastering certain topics. Moreover, there is no verification and

advancement strategy to ensure that learners understand the material covered during lessons.

2.1.4 Personalized Learning Pathways and AI-Guided Academic Improvement in Higher Education [4]

Year: 2026

Dataset(s): Higher education student performance records and learning analytics datasets

The paper presents a study of artificial intelligence-based systems for personalizing educational programs. It aims at modifying the order of delivery and the complexity of learning materials based on the learners' results, relying on learning analytics to detect knowledge gaps and give personalized suggestions. The process involves structured testing and using its results to advance learners through the predefined course curriculum. Analysis over several cohorts of learners reveals significant benefits of adaptive learning pathways over non-adaptive ones in terms of learners' achievements measured by test results, course completion rates, and learners' comprehension levels. The authors stress the significance of transparent modeling of learners' profiles, which would make learners aware of how their progress affects the direction of learning material delivery.

Observations:

The study above strongly validates the efficacy of personalized and adaptive learning pathways – the primary pedagogical technique of SaraswathAI. As evident from the improvements in performance among various groups, the model of mastery learning adopted in SaraswathAI wherein topics are learned based on competency acquired is effective. The concept of transparency of learner modeling also resonates with the SaraswathAI solution that allows for learners to know precisely how much further along the learning journey they have to go. This approach can be used by institutions that want to develop a customized curriculum with pre-created assessments. However, there is one key difference between SaraswathAI and the system described above since SaraswathAI takes away the requirement of manually building knowledge graphs, defining learning paths, and creating assessment quizzes. Instead, all of these are done automatically based on documents uploaded by the learner.

2.1.5 Agent4EDU: Advancing AI for Education with Agentic Workflows [5]

Year: 2025

Dataset(s): Conceptual study based on IoT frameworks, case studies, and prior research

The Agent4EDU paper provides a structured approach to the application of agentic AI workflows to educational activities. According to the paper, there exist four different approaches to the use of AI agents in education. These include the human-AI partnership approach, where the AI agent is used to help the human teacher; the AI assistant approach, where the AI agent helps the student in some way; instruction execution, where the AI agent takes charge of executing a given instructional activity without assistance; and the general type agents approach, where the AI agent is able to execute entire workflows in education. The framework leverages large language models and agentic orchestration pipelines in order to handle various educational tasks such as content curation, tutoring, and test creation.

Observations:

The Agent4EDU model is the closest conceptually to SaraswathAI amongst all the models discussed. This model justifies the architectural choice that the SaraswathAI platform is to be conceived as an agent and not as a retrieval mechanism. The four-model classification is especially helpful since SaraswathAI is more akin to the general type agent where all document processing, knowledge organization, delivering information to the learner, creating assessments, and monitoring the progress toward mastery is handled by one integrated pipeline without any continuous intervention from the instructor. The conclusion arrived at by the researchers that agentic approach helps in performing educational activities beyond the capabilities of single-model agents inspires the architectural decision taken for SaraswathAI, where separate modules such as topic extraction, quiz creation, feedback analysis, and mastery calculations are sequentially linked to form an overall learning agent. Nevertheless, the main drawback of the Agent4EDU paper is that it only proposes an overall model and has no practical implementation that was evaluated on real academic documents for achieving specific levels of mastery.

2.2 Summary of Literature Study

The literature reviewed points to the presence of substantial developments regarding the use of AI in educational processes in aspects such as the usage of agentic AI, the use of content retrievals based on the specificities of particular domains, adaptive learning processes, and intelligent interaction. The analysis of these works leads to a number of valuable insights that will contribute to further work on SaraswathAI.

Among other developments is the growing trend towards the use of agentic AI, where intelligent agents carry out autonomous tasks of content delivery, tutor functions, and assessment processes. The literature provides evidence of the efficiency of self-evolving agents and agentic processes of operation; however, such systems usually have a general nature and are not specialized in educational processes.

Moreover, the use of LLM-based retrieval systems is another promising direction of developing AI technology in education. By using curriculum-specific data sets and advanced retrieval methods, such systems help deliver more relevant answers. Still, while they allow obtaining precise results in information retrieval, these approaches do not solve the problems related to ongoing assessments or personalized processes.

Research on learning interaction and engagement frameworks emphasizes the importance of monitoring student behavior and providing adaptive interventions. These systems improve learner engagement by analyzing interaction patterns and delivering feedback. However, they often operate without integrating structured knowledge representations or guided learning paths, limiting their ability to provide comprehensive learning support.

Further evidence for the value of personalized learning pathways also reinforces the need for an adaptive education system. Research indicates that dynamically adapting the level of difficulty of the content and making suggestions for learners depending on their performance has a high potential of improving the learning process. However, such systems usually lack a link to intelligent information retrieval and learning processes.

Generally, according to the literature, it is evident that although various features such as agency systems, adaptation learning methods, and intelligent retrieval have received

considerable attention, none of them is considered as part of a unified system. In other words, a complete system integrating these features does not seem to exist.

As such, the problem has been addressed through the creation of SaraswathAI which incorporates agentic artificial intelligence, information retrieval through knowledge bases, adaptation assessment, and secure data storage.

Chapter -3 Proposed Method

3.1 Design Methodology

The SaraswathAI system is designed based on the concept of a learning platform comprising document-based content retrieval, modeling of knowledge, adaptive testing, and mastery progression within one closed-loop architecture. Design of the system follows a modular approach, where separate concerns such as document ingestion, modeling of knowledge, delivery of content, creation of test questions, and learner tracking are addressed separately in distinct modules. Each module is responsible for communicating via specific interfaces with other modules involved in the process, which ensures flexibility in future development when changes occur in hardware or models used.

The methodology is based on a five-step learning life cycle. First, data acquisition and preprocessing involve the uploading of academic PDF documents and their extraction of texts using the PyPDF2 library and semantic splitting using the text splitter from LangChain. Second, knowledge modeling includes converting extracted information to vector representations with the help of sentence transformer models that are saved in the FAISS or ChromaDB vector databases, and constructing a Knowledge Graph through the use of NetworkX at the same time. Third, learning path generation encompasses the identification of the conceptually most important topics using the Betweenness Centrality Algorithm as well as generating a studying order by means of the Topological Sort Algorithm. Fourth, content delivery and assessment include topic-oriented RAG retrieval of study explanations for the student in the Study mode and dynamic creation of adaptive quizzes in the Quiz mode through the use of the Adaptive Difficulty Algorithm. Finally, mastery-based progression entails an analysis of quiz results, update of mastery scores for each topic, and determination of whether the user can progress to another topic.

The system operates as a continuous loop rather than a linear sequence — after each quiz cycle, the updated mastery state feeds back into the learning path, potentially reordering priorities or triggering re-engagement with insufficiently understood topics. This feedback architecture ensures that learning is not a one-pass consumption of

content but an iterative, self-correcting process grounded in demonstrated understanding.

Use Case Diagram

The Use Case Diagram captures the functional scope of SaraswathAI from the perspective of its actors and their interactions with the system. The diagram involves three actors and six primary use cases, described below.

Actors:

The Student is the primary actor who interacts with all learning-facing features of the platform. The System acts as an automated actor that executes background processes — document processing, graph construction, quiz generation, and mastery updates — without direct user instruction at each step. The Administrator is a secondary actor responsible for system configuration, model selection, and performance monitoring.

Use Cases:

Upload Document — The Student uploads one or more academic PDF files through the Streamlit interface. The System automatically processes the document, extracts text, generates embeddings, and constructs the Knowledge Graph. This use case is the entry point for all subsequent learning activities and must be completed before any other use case can proceed.

Explore Knowledge Graph — The Student navigates the interactive Knowledge Graph visualisation, examining how topics extracted from the uploaded document are classified and connected. The System renders the graph using the Spring Layout Algorithm and highlights the most important topics identified by Betweenness Centrality. This use case helps the student understand the structure of the subject before beginning formal study.

Follow Learning Path — The Student views the dependency-ordered study sequence generated by the Topological Sort Algorithm. The System presents topics in prerequisite order with individual mastery progress bars. The Student selects topics to

study from this sequence, and the System tracks which topics have been completed, are in progress, or remain to be studied.

Study a Topic — The Student selects a topic and enters Study Mode. The System performs topic-driven RAG retrieval, assembles the most relevant content chunks from the FAISS index, and passes them to the locally hosted LLM to generate a structured explanation. The System also generates code examples and cross-references to related topics. The Student may take personal notes within the interface.

Attempt Quiz — The Student initiates a quiz for a studied topic. The System generates multiple-choice questions using the Quiz Generator module, presents them one at a time, and collects responses. The System evaluates accuracy and response time, applies the Adaptive Difficulty Algorithm to calibrate subsequent question complexity, and updates the topic's mastery score through the Mastery-Based Progression Algorithm upon completion.

Monitor Progress — The Student views the Progress Dashboard, which displays overall completion percentage, per-topic mastery scores, and the distribution of topics across Mastered, In Progress, and To Learn states. The Administrator can access aggregated system performance metrics, model configuration settings, and usage logs through the configuration panel.

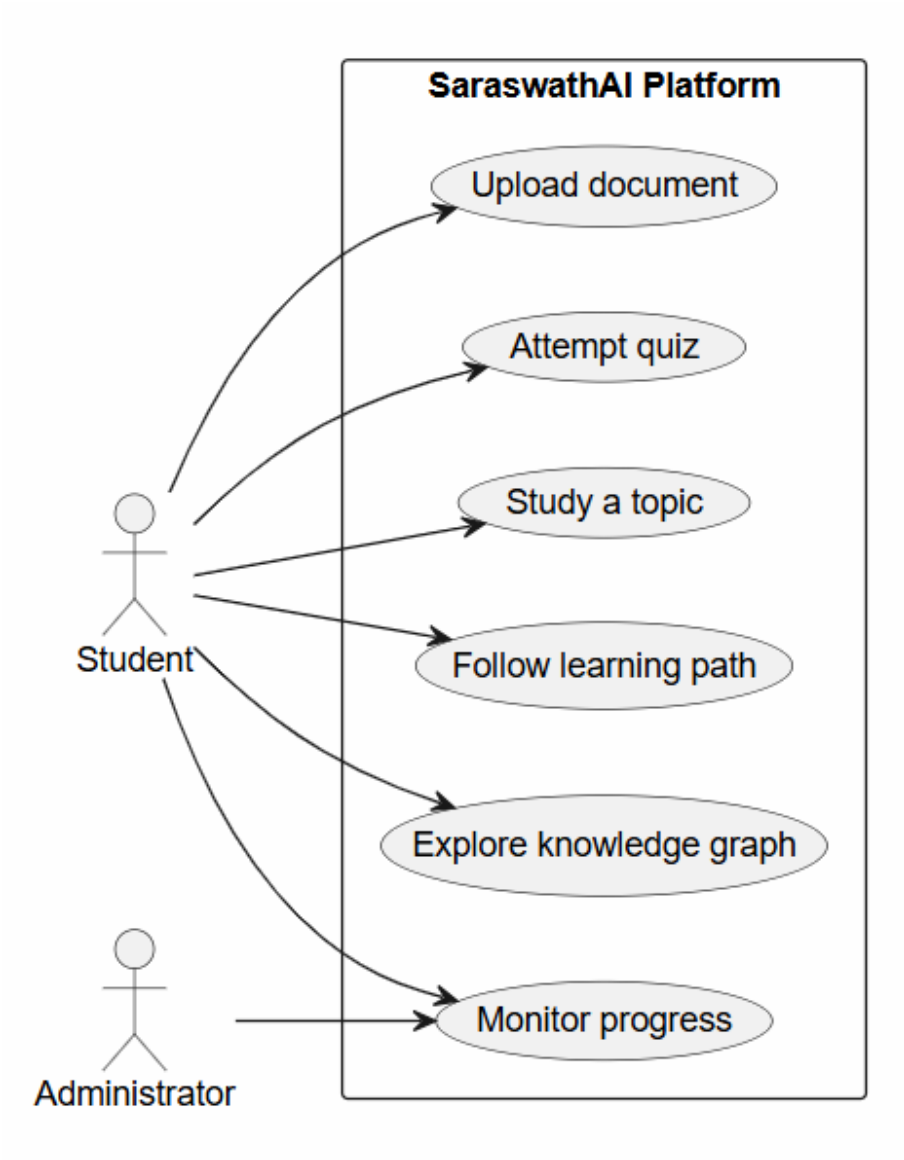

Figure 3.1: Use Case Diagram.

3.2 System Architecture Diagram

The SaraswathAI system architecture is structured in a manner that ensures its functionality as a five-layer modular pipeline for frontend communication, document handling, AI content generation, knowledge modeling, and evaluation. All the modules of this pipeline have specific functions to execute, and the exchange of data between them is done at specific predefined interfaces. The system architecture is depicted in Figure 3.2.

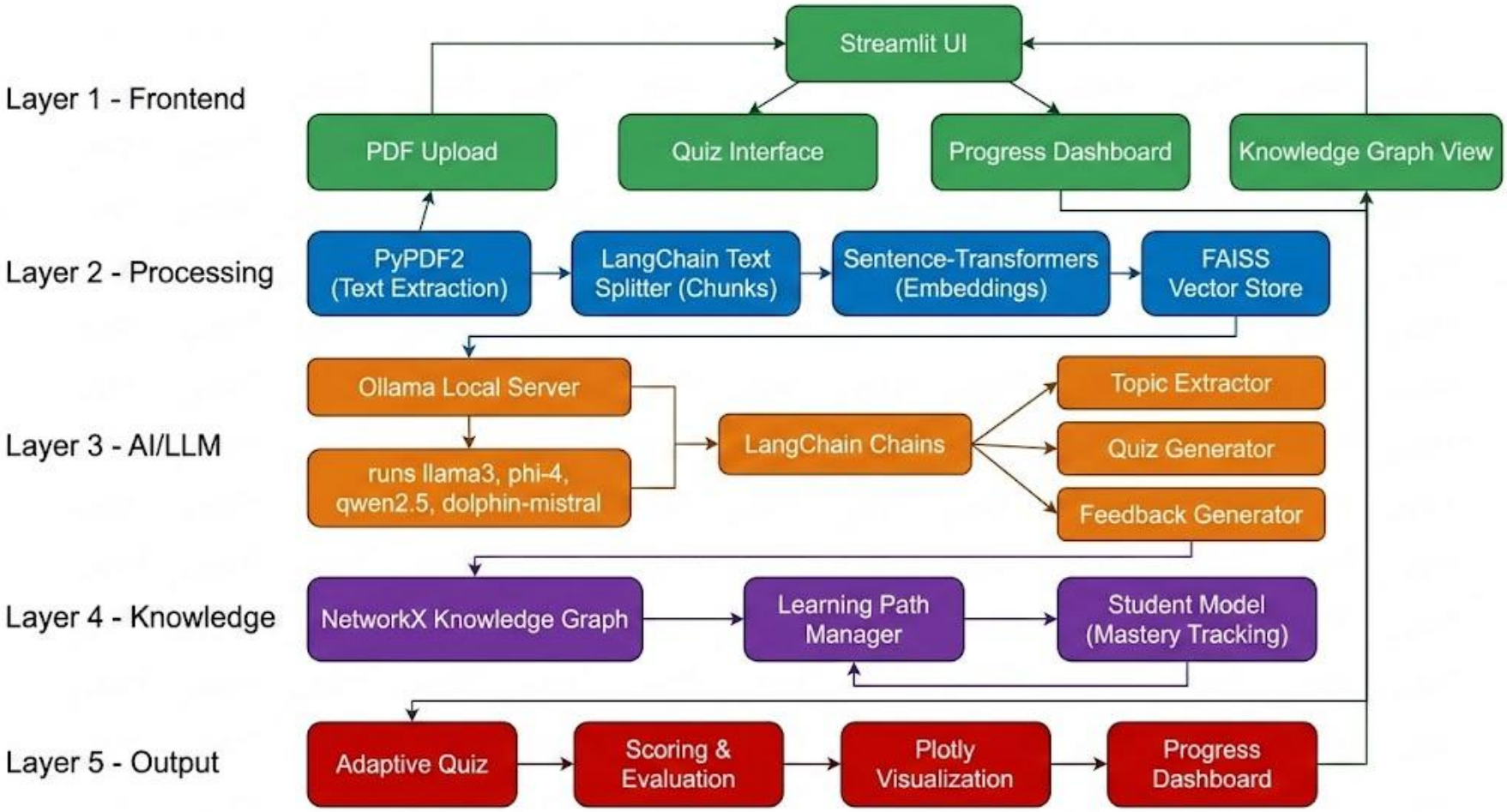

Figure 3.2: Architecture Diagram

The process follows the data movement in a top-to-bottom direction, wherein data moves from the frontend layer to the document processing layer, then to the AI generation layer, after which it goes all the way back to the knowledge layer and the output layer. Nevertheless, the learning pipeline works on a loop as the results from the assessment layer get fed back to the knowledge layer for updating of the mastery scores and even changing the learning path. Such is the two-way communication in the system that sets SaraswathAI apart from a standard retrieval-based solution.

Layer 1: Frontend Layer

The Frontend Layer represents the only interface between the student and the system and is completely developed using Streamlit, a Python-based responsive web framework. It is accountable for presenting all the user interface screens, capturing user input, sending requests to the backend layers, and displaying output as soon as it becomes available.

The layer includes five main interfaces. The PDF Upload Interface allows users to upload files via drag and drop with maximum file size of 200 MB, upload status updates, and initiation of the document processing flow after uploading is completed. The Knowledge Graph Visualization interface shows the NetworkX graph via Plotly, giving students ability to navigate through the graph, filter by topic type, and see details of a particular topic without page refresh. The Learning Path Panel interface shows the list of topics sorted according to their dependency order along with the progress bar showing the level of mastery for each topic after each quiz round. The Study Mode Interface allows students to read AI-generated explanations about topics in a four-tabbed interface (Explanation, Examples, Related Topics, My Notes), with progressive content loading from the LLM to minimize any delay.

All communication between the Frontend Layer and the various backend layers happens through the use of direct Python function calls in the Streamlit runtime environment, while the session state parameters hold the user-specific information like references to uploaded documents, the selected topic, quiz completion status, and level of mastery.

Layer 2: Document Processing Layer

Document processing is performed in the Document Processing Layer. It converts the raw input document uploaded to a structured representation that can be easily searched for retrieving useful knowledge. The entire process takes place automatically after uploading an academic document and generates two main products, namely, a FAISS vector index for semantic search and extracted topics that are used for constructing a knowledge graph, forming a base for further learning.

Text extraction from documents is done through the Text Extraction Module, which utilizes the PyPDF2 library to extract text from PDF documents page-by-page. Formatting features such as headers and footers, which might add noise in the search system, are removed at this stage as well. Extracted text is processed by the Chunking Module, which splits a text document into overlapping pieces of 1,000 tokens, each with a 200-token overlap. Overlapping chunks guarantee that nothing is missed in the retrieval because there might be cases where the same information is distributed between adjacent chunks.

The chunks of text are then transformed in the Embedding Generation Module using GPU accelerated sentence transformer models – all-mpnet-base-v2 for high fidelity embeddings while all-MiniLM-L6-v2 is used when optimization for fast performance is required. This results in a dense embedding of the chunks into a 768 or 384 dimensional vector space depending on the choice of the sentence transformation model. The vectors representing the chunks of text are then stored in the FAISS vector store with the help of an efficient Inverted File Index (IVF) which allows fast nearest neighbour search, even at scale, through tens of thousands of documents in the form of chunks of text. In parallel, the Topic Extraction module feeds the text data of the documents in the LLM layer, giving a prompt indicating that the LLM needs to find out the fundamental concepts, derived concepts and techniques used in the text and their relationships.

Layer 3: AI / LLM Layer

AI Layer is the generative component of the platform, where all natural language outputs (topic explanations, code, quiz questions, and feedback) will be generated based on the information from the document processing layer. This component relies on the

Ollama local inference server to perform the loading, execution, and management of the memory of the locally hosted large language models.

As for the list of supported models within the AI Layer, it includes LLaMA 3.2 and LLaMA 3 for reasoning tasks with multiple steps requiring high coherence; Phi-4 and Phi-3.5 for providing quick responses when inference speed matters more than the depth of the answers; Qwen 2.5 for generating multilingual text and performing cross-domain reasoning; and Dolphin-Mistral for delivering creative answers to students, especially those struggling with certain topics and needing new ways of presenting the information. The advantage of using such models is that they are hosted locally and do not depend on any external APIs.

The orchestration of activities within the functional modules is done via the LangChain chain and prompt management architecture. The Topic Explainer Module creates structured prompts that combine the content chunks with the role of producing explanations on an adequate academic level, followed by an example and a list of related concepts. The prompts are then fed to the model token-by-token and streamed to the frontend. The Quiz Generator Module obtains the content chunks and current difficulty level provided by the Adaptive Difficulty Algorithm, and builds prompts that ask the model to create MCQ questions of a certain level of difficulty featuring one correct response and three wrong ones, in a JSON-formatted way that can be parsed by the system reliably. The Feedback Generator Module analyzes the incorrect answers submitted by the user and generates feedback explaining why a particular answer was incorrect and what part of the original text needs to be reviewed.

Each LLM's response is filtered out by the frontend prior to the display: the content length is verified, the structure is confirmed, and the pattern of refusal is detected to avoid any issues with incorrectly interpreted prompts.

Layer 4: Knowledge and Student Modelling Layer

The Knowledge & Student Modelling Layer is the smart layer which turns raw information contained in documents into structured educational material along with tracking a student's current place in this course. This layer works in two phases: graph construction initiated when a new document is uploaded, and mastering update performed at the end of each quiz cycle.

At the time of graph creation, the Knowledge Graph Manager gets the topics' graph from the Document Processing Layer in the form of an adjacency list of topics and dependencies. Then, using the NetworkX library, it builds a directed graph where each topic corresponds to a certain node, while each dependency becomes a directed edge meaning that one must know a topic before proceeding to its dependents. After that, the Betweenness Centrality Module applies NetworkX's betweenness centrality algorithm on this graph and calculates, for each node, what fraction of the shortest paths between all pairs of nodes pass through it. Nodes with high centrality values are considered high-priority topics – these topics serve as connecting bridges of the whole subject and understanding them will help in comprehending many others.

Topological Sort is performed by the Learning Path Manager on the graph to create a linear sequence of topics that satisfies all the prerequisites among them. In cases where there can be more than one possible ordering, depending on the subject in the actual educational environment with more than one fundamental topic existing independently, the Learning Path Manager resolves the tie based on their centrality scores, giving preference to those with higher scores. This creates a study sequence that is not only accurate in terms of prerequisites but is also strategically oriented. The learning path created is segmented for frontend presentation.

Student Model runs on a continuous basis during the whole process of learning. Every time the user takes the quiz, it gets the user's correctness percentage and average answer time from the frontend and runs the Mastery-Based Progression Algorithm to calculate the new topic mastery score. The topic with a higher mastery score than the predetermined threshold value (by default, 80%) becomes a Mastered topic and is highlighted in the Knowledge Graph view and Learning Path panel. The topic with a lower mastery score becomes an In Progress topic, which is added back into the queue so that the user cannot proceed to other dependent topics until mastering the topic. The session progress percentage that is shown consistently in the sidebar is calculated by dividing the number of Mastered topics by the total number of topics in the graph.

Layer 5: Assessment and Output Layer

The role of the Assessment and Output Layer includes adaptive quiz delivery, assessment response analysis, and results presentation in order to complete the learning

cycle started by the Study Mode. The Assessment and Output Layer does not create any content on its own; rather, it coordinates the output of the other two layers mentioned earlier.

The Adaptive Quiz Module controls the life cycle of the quiz. The module assigns an initial level of difficulty for the quiz, depending on the mastery score of the student for the chosen topic; a student with no previous attempt at the quiz starts at a medium level, whereas a student with previous attempts but unable to pass the mastery score starts at an easy level. Once the answer is given, the Adaptive Difficulty Algorithm takes the score and time taken into account, adjusting the next question difficulty based on the answers provided: the correct answer given swiftly will have a one-step increase in difficulty; the correct answer given slowly will retain the same level of difficulty, and the wrong answer will lower the difficulty level by one step within the range of the lowest and highest levels of difficulty.

The Scoring and Evaluation Module gathers the entire answers after completing the quiz and calculates the accuracy score and average time taken to answer questions and provides the metrics to the Student Model in Layer 4 for updating the mastery score. Also, it analyzes the questions that have been answered incorrectly and extracts the corresponding chunk of source data from the Vector Store for providing customized feedback using the Feedback Generator in Layer 3. This feedback will be provided to the learner in conjunction with the quiz scores.

The Progress Dashboard combines the metrics obtained from the Student Model and Assessment Module in one dashboard displaying the overall learning status of the student based on all dimensions at once such as overall session progress, topic-wise mastery scores, distribution of topics in terms of topic types from knowledge graph, and ranking of most critical topics based on betweenness centrality measure. Here, Plotly Visualization Module generates the graph in the form of a bar chart for topic distribution and also generates the knowledge graph in the form of a force-directed network diagram.

3.3 Description of Algorithms

3.3.1 Semantic Retrieval using FAISS

FAISS (Facebook AI Similarity Search) uses L2 distance-based Approximate Nearest Neighbor search to efficiently retrieve the most semantically similar document chunks for a given query.

Pseudocode:

Input: Query text Q, FAISS Index I, Sentence Transformer Model M

Output: Top-k relevant document chunks

1. Tokenize and encode Q using Model M to produce query vector q_vec.
2. Load FAISS Index I (IVF structure with pre-indexed chunk vectors).
3. For each cluster in I, compute candidate vectors within nearest Voronoi cells.
4. Calculate L2 distance: $dist(q_vec, chunk_vec) = \sqrt{\Sigma(q_i - c_i)^2}$
5. Sort all candidate distances in ascending order.
6. Select the top-k chunk vectors with the smallest distances.
7. Retrieve the original text chunks corresponding to the top-k vectors.
8. Return the top-k text chunks as retrieval results.

3.3.2 Recursive Text Segmentation

Recursively splits documents into semantically coherent chunks within the token limit, ensuring no context is lost across segment boundaries.

Pseudocode:

Input: Document text D, Max token limit L, Overlap size O

Output: List of overlapping text chunks C

1. If length(D) ≤ L:

Return [D] as a single chunk.

2. Attempt split at double newline boundaries (paragraph level).
3. If segments still exceed L:

   Attempt split at single newline boundaries (sentence level).

4. If segments still exceed L:

   Split at whitespace boundaries (word level).

5. For each resulting segment S:

   If length(S) > L:

   Recursively apply steps 2–4 on S.

6. Apply overlap: for adjacent chunks C[i] and C[i+1],

   append the last O tokens of C[i] to the start of C[i+1].

7. Tag each chunk with source document ID and position metadata.
8. Return final list of chunks C.

3.3.3 BM25-Based Retrieval

BM25 ranks document chunks by term frequency and inverse document frequency, with saturation and length normalization, to complement semantic FAISS search with keyword-level precision.

Pseudocode:

Input: Query Q, Corpus of chunks {D1, D2, ..., Dn}, Parameters k1, b

Output: Ranked list of chunks by BM25 score

1. Tokenize Q into query terms: q = [q1, q2, ..., qm].
2. For each chunk Di:

   Tokenize Di into terms.

Compute TF(qi, Di) = frequency of term qi in Di.

Compute IDF(qi) = log((N − df(qi) + 0.5) / (df(qi) + 0.5) + 1)

where N = total chunks, df(qi) = chunks containing qi.

3. For each chunk Di, compute BM25 score:

score(Di, Q) = Σ [IDF(qi) × (TF(qi,Di) × (k1+1)) /

(TF(qi,Di) + k1 × (1 − b + b × |Di|/avgdl))]

where avgdl = average chunk length in tokens.

4. Sort all chunks by score in descending order.
5. Return the top-k highest scored chunks.

3.3.4 Autoregressive Answer Generation

The LLM generates answers token-by-token in an autoregressive fashion, conditioned on the verified retrieved context and the student's query.

Pseudocode:

Input: Retrieved context chunks C, Student query Q, LLM Model LM

Output: Generated answer text A

1. Assemble prompt P:

P = [System Instruction] + [Context from C] + [Query Q] + [Generation directive]

2. Tokenize P into input token sequence T_in.
3. Initialize output token sequence T_out = [].
4. Repeat until stopping condition:

a. Feed T_in + T_out to LM.

b. Compute probability distribution over vocabulary for next token.

c. Sample or argmax-select next token t_next.

d. Append t_next to T_out.

e. If t_next == EOS token OR len(T_out) ≥ MAX_TOKENS:

Break.

5. Detokenize T_out to produce answer text A.
6. Stream A token-by-token to the frontend interface.
7. Return A.

3.3.5 SHA-256 Cryptographic Hashing

SHA-256 generates a unique, fixed-length 256-bit fingerprint for each document chunk, which is stored immutably on the blockchain to detect any tampering.

Pseudocode:

Input: Document text chunk T

Output: 256-bit hash string H

1. Convert T to byte array B.
2. Pad B to 448 mod 512 bits:

   Append bit '1', then '0' bits, then 64-bit length of original B.

3. Initialize eight 32-bit hash values H0–H7 from fractional parts

   of square roots of the first 8 prime numbers.

4. Divide padded B into 512-bit message blocks M[0..n].
5. For each block M[i]:

   a. Expand M[i] into a 64-word message schedule W[0..63].

   b. Initialize working variables a,b,c,d,e,f,g,h = H0..H7.

   c. For round r = 0 to 63:

Apply bitwise rotations, XOR, AND, NOT operations.

Update working variables using W[r] and round constant K[r].

d. Add working variables back to H0..H7.

6. Concatenate H0..H7 into a 256-bit value.
7. Convert to 64-character hexadecimal string H.
8. Return H.

3.3.6 ECDSA (Elliptic Curve Digital Signature Algorithm)

ECDSA authenticates all blockchain transactions using elliptic curve cryptography, ensuring only authorized entities can write document hashes to the distributed ledger.

Pseudocode:

Input: Transaction data TX, Private key d, Curve parameters (G, n, secp256k1)

Output: Digital signature (r, s)

KEY GENERATION (once at startup):

1. Select random private key d from [1, n−1].
2. Compute public key Q = d × G (elliptic curve point multiplication).
3. Store (d, Q) for signing and verification.

SIGNING:

4. Hash TX using SHA-256 to produce digest e.
5. Extract z = leftmost bits of e (matching bit length of n).
6. Generate cryptographically secure random nonce k from [1, n−1].
7. Compute point (x1, y1) = k × G.
8. Compute r = x1 mod n. If r == 0, go to step 6.
9. Compute $s = k^{-1} \times (z + r \times d) \bmod n$. If s == 0, go to step 6.
10. Return signature (r, s).

VERIFICATION (by any party with public key Q):

11. Compute $w = s^{-1} \bmod n$.
12. Compute $u1 = z \times w \bmod n$, $u2 = r \times w \bmod n$.
13. Compute point $(x1, y1) = u1 \times G + u2 \times Q$.
14. If $r \equiv x1 \bmod n$: Signature is VALID. Else: INVALID.

3.3.7 ABI Parameter Encoding Algorithm

ABI encoding translates Python smart contract function calls into EVM-compatible bytecode by standardizing function selectors and parameter serialization.

Pseudocode:

Input: Function signature FS (e.g., "storeDocumentHash(bytes32,string,uint256)"),

Parameter list PARAMS

Output: Encoded calldata string CD

1. Compute function selector:

   selector = Keccak256(FS)[0:4 bytes] → 8 hex characters.

2. Classify each parameter in PARAMS:

   Static types → uint256, address, bool, bytes32

   Dynamic types → string, bytes, dynamic arrays

3. For each static parameter P:

   Encode P as 32-byte left-zero-padded hex value.

   Append to static section SS.

4. For each dynamic parameter P:

   Calculate offset = current position in calldata.

   Append 32-byte offset pointer to SS.

   Encode length of P as 32-byte value → append to dynamic section DS.

Encode P data padded to 32-byte boundary → append to DS.

5. Assemble final calldata:

   CD = "0x" + selector + SS + DS

6. Return CD.

3.3.8 Gas Estimation and Optimization Algorithm

This algorithm predicts the Ethereum computational cost per transaction and optimizes gas pricing to ensure reliable, cost-effective blockchain operations.

Pseudocode:

Input: Transaction calldata CD, Network conditions NET

Output: Estimated gas G_est, Optimized gas price GP_opt

GAS ESTIMATION:

1. Set base_gas = 21,000 (fixed per transaction).
2. For each byte in CD:

   If byte == 0x00: data_cost += 4 gas.

   Else: data_cost += 16 gas.

3. Compute execution_cost by summing EVM opcode costs:

   New storage slot write → 20,000 gas each.

   Existing slot update → 5,000 gas each.

   Other opcodes → standard EVM cost table.

4. total_gas = base_gas + data_cost + execution_cost.
5. Add safety buffer: G_est = total_gas × 1.10 (10% margin).

GAS PRICE OPTIMIZATION:

6. Query NET to obtain base_fee_per_gas (EIP-1559).

7. Set priority_multiplier based on urgency:

   Normal → 1.0×, High → 1.5×, Critical → 2.0×.

8. GP_opt = base_fee_per_gas × priority_multiplier.
9. Compute transaction cost in ETH:

   cost_wei = G_est × GP_opt

   cost_eth = cost_wei / 10^{18}

10. Return G_est, GP_opt, cost_eth.

3.3.9 Betweenness Centrality Algorithm

Betweenness Centrality identifies the most conceptually important topics in the Knowledge Graph by measuring how frequently each topic lies on the shortest path between all other topic pairs.

Pseudocode:

Input: Directed Knowledge Graph G = (V, E)

Output: Dictionary BC mapping each node v to its centrality score

1. Initialize BC[v] = 0.0 for all v in V.
2. For each source node s in V:

   a. Run BFS/Dijkstra from s to compute:

      S = stack of nodes in order of non-decreasing distance from s.

      P[v] = list of predecessors of v on shortest paths from s.

      sigma[v] = number of shortest paths from s to v.

   b. Initialize delta[v] = 0.0 for all v.

   c. While S is not empty:

      w = pop from S.

For each predecessor v in P[w]:

delta[v] += (sigma[v] / sigma[w]) × (1 + delta[w])

If w ≠ s:

BC[w] += delta[w]

3. Normalize BC values:

   For undirected G: BC[v] = BC[v] / ((n−1) (n−2)/2)

   For directed G: BC[v] = BC[v] / ((n−1) (n−2))

   where n = |V|.

4. Sort nodes by BC score in descending order.
5. Return BC dictionary (high-score nodes = critical bridge topics).

3.3.10 Topological Sort Algorithm (Kahn's Algorithm)

Topological Sort orders Knowledge Graph topics so that every prerequisite topic appears before the topics that depend on it, generating the correct study sequence.

Pseudocode:

Input: Directed Acyclic Graph G = (V, E)

Output: Linearly ordered list TOPO_ORDER respecting all dependencies

1. Initialize IN_DEGREE[v] = 0 for all v in V.
2. For each directed edge (u → v) in E:

   IN_DEGREE[v] += 1.

3. Initialize queue Q.

   For each vertex v in V:

   If IN_DEGREE[v] == 0:

   Enqueue v into Q. (v has no prerequisites)

4. Initialize empty list TOPO_ORDER.
5. While Q is not empty:

   u = Dequeue from Q.

   Append u to TOPO_ORDER.

   For each neighbor v of u (i.e., edge u → v exists):

   IN_DEGREE[v] -= 1.

   If IN_DEGREE[v] == 0:

   Enqueue v into Q.

6. If len(TOPO_ORDER) ≠ |V|:

   Report error: "Cycle detected in prerequisite graph."

7. Break TOPO_ORDER into depth segments:

   Segment 1 = topics with no prerequisites.

   Segment k = topics whose prerequisites are all in Segments 1...(k−1).

8. Apply Betweenness Centrality as tiebreaker within each segment.
9. Return segmented TOPO_ORDER as the Learning Path.

3.3.11 Force-Directed Layout Algorithm (Spring Layout)

The Fruchterman-Reingold spring layout algorithm positions Knowledge Graph nodes in 2D space so that related topics cluster visually together for intuitive exploration.

Pseudocode:

Input: Graph G = (V, E), Iterations MAX_ITER

Output: 2D position dictionary position[v] for all v in V

1. Assign random initial positions: position[v] = random (x, y) for all v in V.

2. Set optimal distance $k = 1 / \sqrt{|V|}$.
3. Set initial temperature T = initial movement limit.
4. Repeat for iter = 1 to MAX_ITER:

   a. Reset displacement: disp[v] = (0, 0) for all v in V.

   b. REPULSIVE FORCES (all node pairs):

      For each pair (u, v) where $u \neq v$:

      delta = position[v] − position[u].

      dist = ||delta||.

      force = k^2 / dist.

      disp[v] += (delta / dist) × force.

      disp[u] −= (delta / dist) × force.

   c. ATTRACTIVE FORCES (connected edges only):

      For each edge (u, v) in E:

      delta = position[v] − position[u].

      dist = ||delta||.

      force = $dist^2$ / k.

      disp[v] −= (delta / dist) × force.

      disp[u] += (delta / dist) × force.

   d. UPDATE POSITIONS:

      For each v in V:

      move = min(||disp[v]||, T).

      position[v] += (disp[v] / ||disp[v]||) × move.

e. COOL DOWN: T = T × cooling_factor. (Reduce T each iteration)

5. Return position dictionary.

3.3.12 Mastery-Based Progression Algorithm

Tracks and updates per-topic mastery scores after each quiz, using a weighted average that prioritises recent performance, and unlocks dependent topics only after the mastery threshold is crossed.

Pseudocode:

Input: Learner L, Ordered topic list S = [s1, s2, ..., sn], Mastery threshold T (default 80%)

Output: Mastery status for all topics; unlocked next topics

1. Initialize mastery_score[s] = 0.0 and attempt_count[s] = 0 for all s in S.
2. For each topic s in S (following Learning Path order):

   While mastery_score[s] < T:

   a. Present study content for s via Study Mode (RAG retrieval).

   b. Generate and deliver quiz for s via Quiz Mode.

   c. Collect performance result R = accuracy score (0.0 to 1.0).

   d. Increment attempt_count[s] += 1.

   e. Update mastery score (weighted toward recent attempts):

   weight = attempt_count[s] / (attempt_count[s] + 1)

   mastery_score[s] = (weight × mastery_score[s])

   + ((1 − weight) × R)

   f. Provide targeted feedback for incorrect answers.

   g. Adjust quiz difficulty via Adaptive Difficulty Algorithm.

End While

Mark topic s as MASTERED in Knowledge Graph.

Unlock all dependent topics of s in the Learning Path.

3. Update Progress Dashboard with current mastery_score[s] for all s.
4. Return updated mastery status map.

3.3.13 Adaptive Difficulty Algorithm

Dynamically adjusts the difficulty level of quiz questions after each answer based on the student's accuracy and response time, keeping challenges calibrated to demonstrated ability.

Pseudocode:

Input: current_difficulty (integer 1–10),

user_performance (float 0.0–1.0, recent accuracy),

response_time (seconds),

target_performance (default 0.75)

Constants: INCREASE_STEP=1, DECREASE_STEP=1,

MIN_DIFFICULTY=1, MAX_DIFFICULTY=10,

FAST_THRESHOLD=15s, SLOW_THRESHOLD=30s

Output: updated_difficulty (integer 1–10)

1. Set difficulty = current_difficulty.
2. ACCURACY-BASED ADJUSTMENT:

If user_performance > target_performance:

difficulty += INCREASE_STEP. (Student performing well → harder)

Else if user_performance < target_performance:

difficulty −= DECREASE_STEP. (Student struggling → easier)

Else:

difficulty = current_difficulty. (On target → no change)

3. SPEED-BASED ADJUSTMENT:

   If response_time < FAST_THRESHOLD:

   difficulty += INCREASE_STEP. (Answered too quickly → increase)

   Else if response_time > SLOW_THRESHOLD:

   difficulty −= DECREASE_STEP. (Answered too slowly → decrease)

4. BOUNDARY CLAMPING:

   If difficulty > MAX_DIFFICULTY: difficulty = MAX_DIFFICULTY.

   If difficulty < MIN_DIFFICULTY: difficulty = MIN_DIFFICULTY.

5. Pass updated difficulty to Quiz Generator prompt as explicit

   instruction: "Generate a [difficulty-label] question."

   Levels 1–3 → "easy (definition recall)"

   Levels 4–6 → "medium (application/analysis)"

   Levels 7–10 → "hard (multi-step reasoning)"

6. Return updated_difficulty.

3.4 Description of datasets, Requirements and Tools

3.4.1 Datasets

This system runs on data sets that are dynamic and can be used for the purpose of knowledge representation and retrieval. The data sets are dynamic and continue to grow as the system is being used by the users.

- **Educational Content Corpus:**

The main data source comprises user uploaded educational documents that are PDF files. The documents encompass textbooks, notes, reference material, and study material related to particular subjects in accordance with course specifications. Documents in PDF format are processed through the use of PyPDF2 module in order to extract text from each document and the extracted text is further broken down to smaller text chunks with the help of Langchain Text Splitter. The resulting processed chunks remain semantically coherent and are the basic input for further processing.

- **Semantic Embedding Dataset:**

Once the preprocessing stage is completed, each textual chunk undergoes transformation into vector representations through the use of Sentence-Transformers models, including all-mpnet-base-v2 and all-MiniLM-L6-v2. By representing the data in vectors, the models enable finding relations between different components of the 29 content semantically and contextually rather than by keywords. Vectors can be used as inputs to search systems built with vector databases such as FAISS or ChromaDB.

- **Knowledge Graph Dataset:**

The structured data set is built in the form of a Knowledge Graph based on NetworkX where the nodes will be the extracted topics while the links between the nodes will indicate any kind of relationship or dependency between the topics. The structured data set is created from the processed document and it is updated every time a new document is added.

- **User Interaction and Performance Data:**

The system will maintain a dynamic database consisting of the interactions between learners, such as their quiz answers, performance score, and progression level. These pieces of information are employed in computing mastery levels and monitoring the learning process using the Student Model. They also enable adaptive processes, including the ability to change the difficulty level and provide feedback.

3.4.2 Requirements and Tools

To implement the proposed architecture, specific tools and hardware are required to handle both the heavy computational load of AI and the cryptographic operations of Blockchain.

Table 3.1: Software Requirements

Component	Technology/Library	Purpose
Programming Language	Python	Core development of the system, including backend logic, AI integration, and data processing
LLM Framework	LangChain	Builds LLM-powered workflows for content retrieval, quiz generation, and feedback mechanisms
Document Processing	PyPDF2	Extracts text from uploaded PDF documents for knowledge base creation
Graph Processing	NetworkX	Constructs and analyzes knowledge graphs for learning path generation
Visualization	Plotly	Provides interactive visualization of knowledge graphs and learning structures
Embedding Models	sentence-transformers (all-mpnet-base-v2, all-MiniLM-L6-v2)	Converts text into vector embeddings for semantic search
Vector Database	FAISS / ChromaDB	Stores and retrieves embeddings efficiently for similarity-based content retrieval
Backend Model Serving	Ollama	Hosts and manages LLMs locally such as LLaMA, Phi, and Mistral models

Frontend Framework	Streamlit	Develops an interactive user interface for the application

Table 3.2: Hardware Requirements

Component	Specification	Rationale
Processor (CPU)	Intel i5 / AMD Ryzen 5 or higher	Handles general computation, backend processing, and system operations efficiently
Graphics Processing Unit (GPU)	NVIDIA GPU with CUDA support (optional but recommended)	Accelerates LLM inference and embedding computations for faster performance
RAM	Minimum 8 GB (16 GB recommended)	Supports smooth execution of AI models, vector databases, and multiple processes
Storage	Minimum 20 GB free space (SSD preferred)	Stores datasets, models, embeddings, and application files

Chapter -4 Results and Observations

4.1Stepwise description of Results

The below steps outline the entire flow for executing SaraswathAI starting from its initialization until the end where the student completes the quiz, according to the results generated after using the CMS project document (185 KB PDF). The below steps outline exactly what was done, what outputs were generated by the system, and how this ties to the interfaces in the result figures 4.2.

Step 1: System Initialization and Environment Setup

The system starts up using the Streamlit GUI and opens up the Setup & Configuration pane from the left sidebar. At startup, the backend checks for the availability of hardware and successfully detects that an NVIDIA GeForce RTX 3050 Ti Laptop GPU with 4.0 GB VRAM can be utilized, evident from the green status message in the left-hand pane stating "GPU detected: NVIDIA GeForce RTX 3050 Ti Laptop GPU (4.0 GB)." The LLM model is initialized using llama3.2:latest, while the embeddings use sentence-transformers/all-MiniLM-L6-v2, with GPU optimization turned on. The presence of the Ollama backend is verified, and the FAISS index vector store is set up for indexing. With all the prerequisites completed, the application renders the four cards of the four modes: Knowledge Graph, Learning Path, Study Mode, and Quiz Mode, indicating successful startup of the system, as demonstrated in Figure 4.1.

Step 2: Document Upload and Text Extraction

Now that the system has been initialized, the user uploads an academic PDF file using the "Upload Learning Materials" widget in the sidebar panel. During the test, the document for a CMS project, weighing 185 KB, was uploaded. Clicking on "Process Textbooks," the PyPDF2 library starts extracting text from the PDF, eliminating all formatting elements like headers, footers, and pagination to generate plain text. This process shows its status in real-time using the Streamlit UI.

Step 3: Text Chunking and Preprocessing

The selected text is divided into overlapping pieces using LangChain's RecursiveCharacterTextSplitter, where the chunk size is roughly 1,000 tokens and there is an overlap of 200 tokens between chunks. The overlap helps ensure that any concept

that runs across paragraph breaks is contained entirely in one retrievable chunk. Each chunk is then tagged with an ID and associated with its source document information. These chunks serve as the basis for the semantic embedding dataset and will be inputted into both the embedding creation function and the topic extraction function.

Step 4: Embedding Generation and Vector Storage

The chunk of text is transformed into a dense vector through the sentence-transformers/all-MiniLM-L6-v2 model, which creates a vector of size 384 that carries the semantics of the input. GPU computation with CUDA is used here to minimize the time it takes to compute the embeddings of all the chunks of the provided file, which would otherwise take much longer on CPUs alone. These created vectors are indexed in a vector store of FAISS with the help of Inverted File Indexes for faster approximate nearest neighbour search. A mapping between the generated embeddings and their corresponding chunks of text is also stored to be able to show the text chunks themselves later.

Step 5: Knowledge Graph Construction and Visualization

Concurrently with the embedding process, the LLM also processes the extracted text to classify the academic topics into three categories as Fundamental Concept, Derived Concept, and Technique/Method, and derives the relationship among them based on their dependency. This classification information is then used by NetworkX to form a directed Knowledge Graph. For the test document, which was taken from CMS project, 14 topics belonging to all three categories were classified and visualized in a graph with different types of nodes such as Fundamental, Derived, and Technique, as depicted in the Knowledge Graph window. In the Spring Layout (Fruchterman-Reingold), the position of each node is calculated using physical attraction and repulsion force, resulting in a physically interpretable layout with related nodes clustered together. At the beginning of the session, as seen in Figure 4.2, it is displayed that there are 0 Mastered, 0 In Progress, and 14 To Learn.

Step 6: Learning Path Generation

The Learning Path Manager makes use of Kahn's Topological Sorting algorithm, using the Knowledge Graph constructed above to create a study path that respects all dependencies among the 14 extracted topics. This learning path is segmented based on

the degree of prerequisites; topics with zero prerequisites belong to Segment 1, while those whose prerequisites all come from Segment 1 belong to Segment 2. During the test session, the first segment included five topics, namely User Authentication, Incident Management, Risk Management, Prisma Schema Design, and Database Indexing Strategy, whereas the second segment started with Topic 6, Authentication Header, as shown in Fig. 4.3. For each concept along the path, a mastery progress bar is displayed with an initial value of 0.0%, and there is a Study button leading to Study Mode. Betweenness Centrality scores are calculated throughout the entire graph, and in case of conflicts, it is used as a tie breaker.

Step 7: Study Mode — Content Retrieval and Explanation Generation

Once the student picks a topic from the Learning Path and activates Study Mode, a topic-based FAISS similarity search is conducted to pull out the most appropriate chunks of information about the topic within the indexed document. These chunks of information are compiled into an explanation prompt for use with LLaMA 3.2 model run locally with the help of Ollama that will create a complete explanation of the topic using only information retrieved from the source document. On the topic of User Authentication, the topic-based retrieval yielded a structured topic explanation named "User Authentication: A Comprehensive Overview" containing the importance of authentication, advantages of authentication in security, privacy, and trust aspects, and authentication process walkthrough as shown in Figure 4.4 below. This explanation is gradually presented in the interface, token by token, to prevent any delay in presentation. The Study Mode interface categorizes the information received into four tabs namely Explanation, Examples, Related Topics, and My Notes.

Step 8: Code Example Generation

Under the "Examples" tab, the program creates a realistic piece of code related to the topic from the retrieved information used to create the description. In the case of "User Authentication," an LLM generated a Login.tsx React component that covered a whole login process, including user authentication through the POST method at /auth/login, handling the JWT tokens, and updating the AuthContext, as seen in Figure 4.5. This proves that the tool's capacity to generate information is not limited to explanatory text but also includes creating technical code grounded in documentation.

Step 9: Quiz Generation and Adaptive Assessment

Having finished Study Mode on the selected topic, the learner progresses to Quiz Mode, in which the Quiz Generator selects the same content pieces employed during the study session and creates multiple-choice questions according to a set prompt for the large language model (LLM). The prompt outlines the type of question needed, the difficulty level, and the quality of distractors. Each quiz contains 5 multiple-choice questions that are displayed in turn without distractions. Specifically, the screen displays the number of the current question, the topic name, and the available answers (Figure 4.6). In the test quiz, the first question was: "Which of these is NOT a common embedding model?" The possible answers were A. BERT, B. GPT-4, C. Sentence Transformers, and D. FastText — a direct reflection of the document content used for the practice phase. The Adaptive Difficulty Algorithm analyzes student performance (i.e., their accuracy and response time) and adapts the difficulty of the next question appropriately.

Step 10: Mastery Tracking and Progress Visualization

The Scoring and Evaluation Module determines the accuracy rate and average time per question after completing the quiz and submits both values to the Mastery-Based Progression Algorithm, which performs a weighted update of the mastery rate for that topic. Topics with mastery rates exceeding 80% become Mastered; their corresponding nodes are flagged accordingly on the Knowledge Graph, and the next topics on the learning path are unlocked. Topics with lower mastery rates are tagged as In Progress and added to the list for revision until they meet the threshold value. The Progress Dashboard calculates the average mastery rate per topic and displays it as the progress rate in the sidebar. The distribution of topics by type and the list of the most important topics, ranked according to Betweenness Centrality Score, as shown in Figure 4.7, allow students to visualize their standing in the course and the importance of each subject. See Figure 4.8.

4.2 Result Analysis

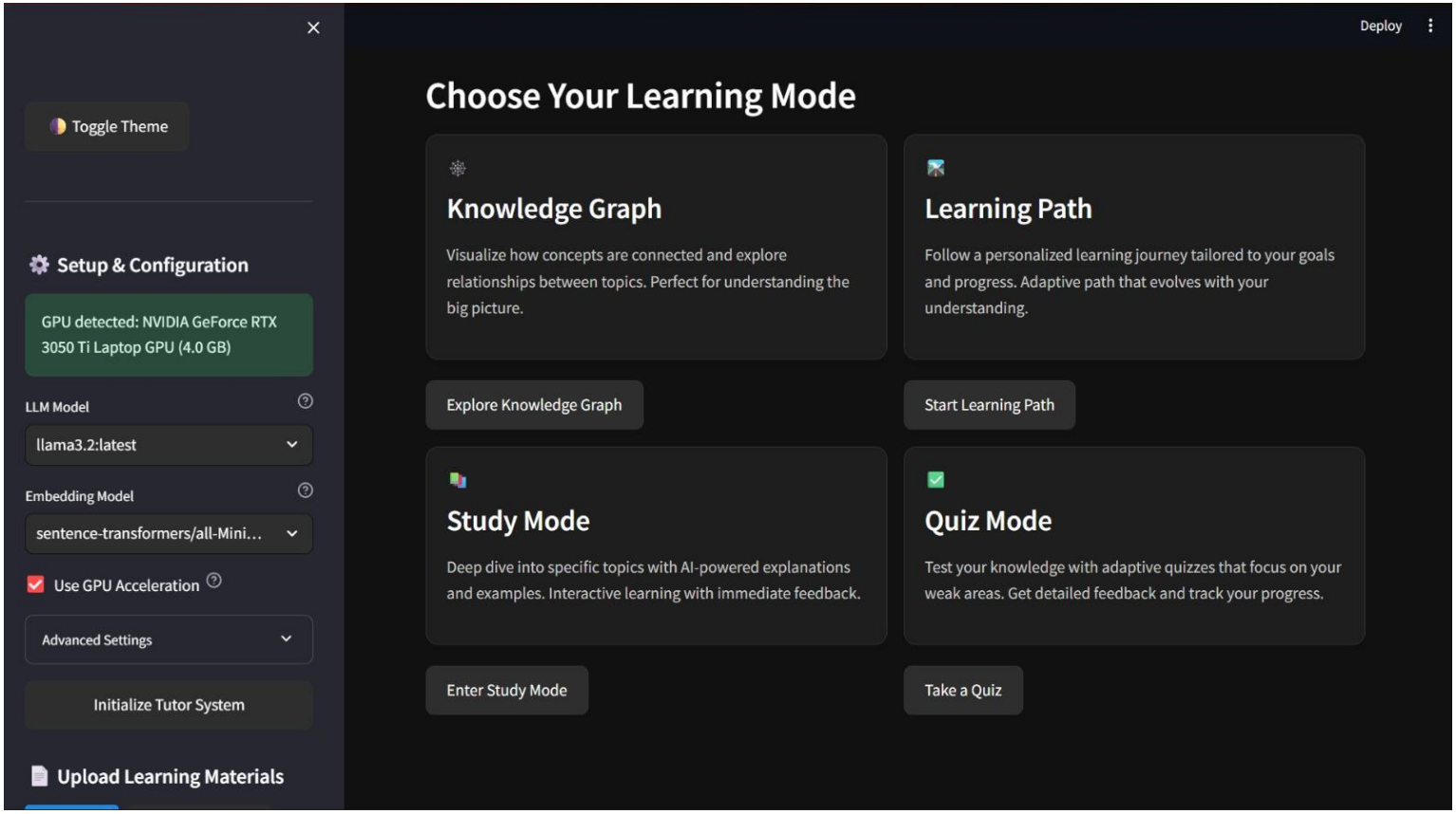

Figure 4.1: Main Dashboard – Learning Mode Selection Interface

The main dashboard presents four learning modes — Knowledge Graph, Learning Path, Study Mode, and Quiz Mode — as interactive cards. The sidebar confirms successful GPU detection and model initialization, indicating the system is ready for a student session.

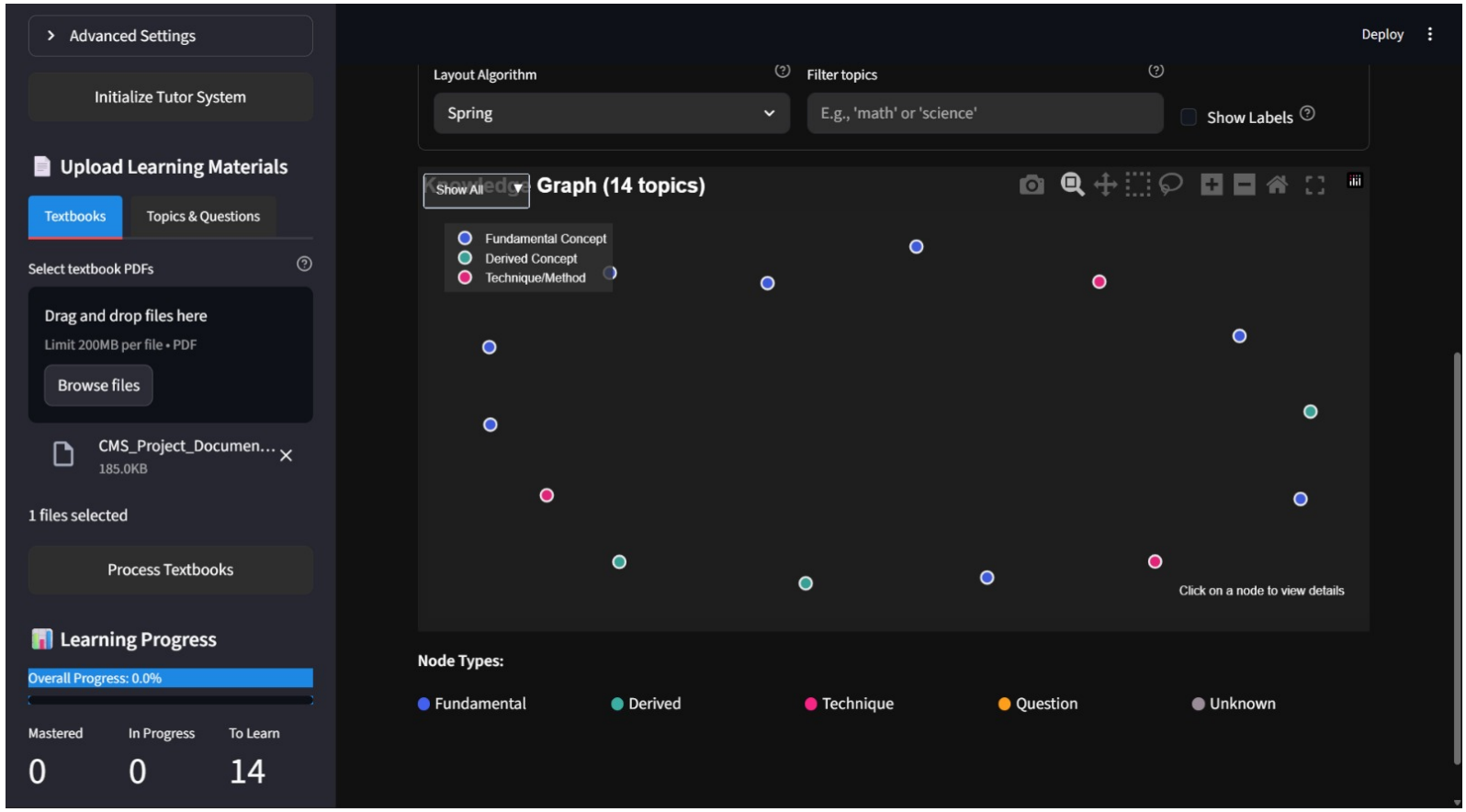

Figure 4.2: Knowledge Graph – Topic Visualization After Document Processing

After processing the uploaded PDF, the system constructed a Knowledge Graph of 14 topics, categorized into Fundamental Concepts (blue), Derived Concepts (green), and Techniques/Methods (red). The Spring Layout Algorithm distributes nodes spatially, with the sidebar tracking all 14 topics as "To Learn" at session start.

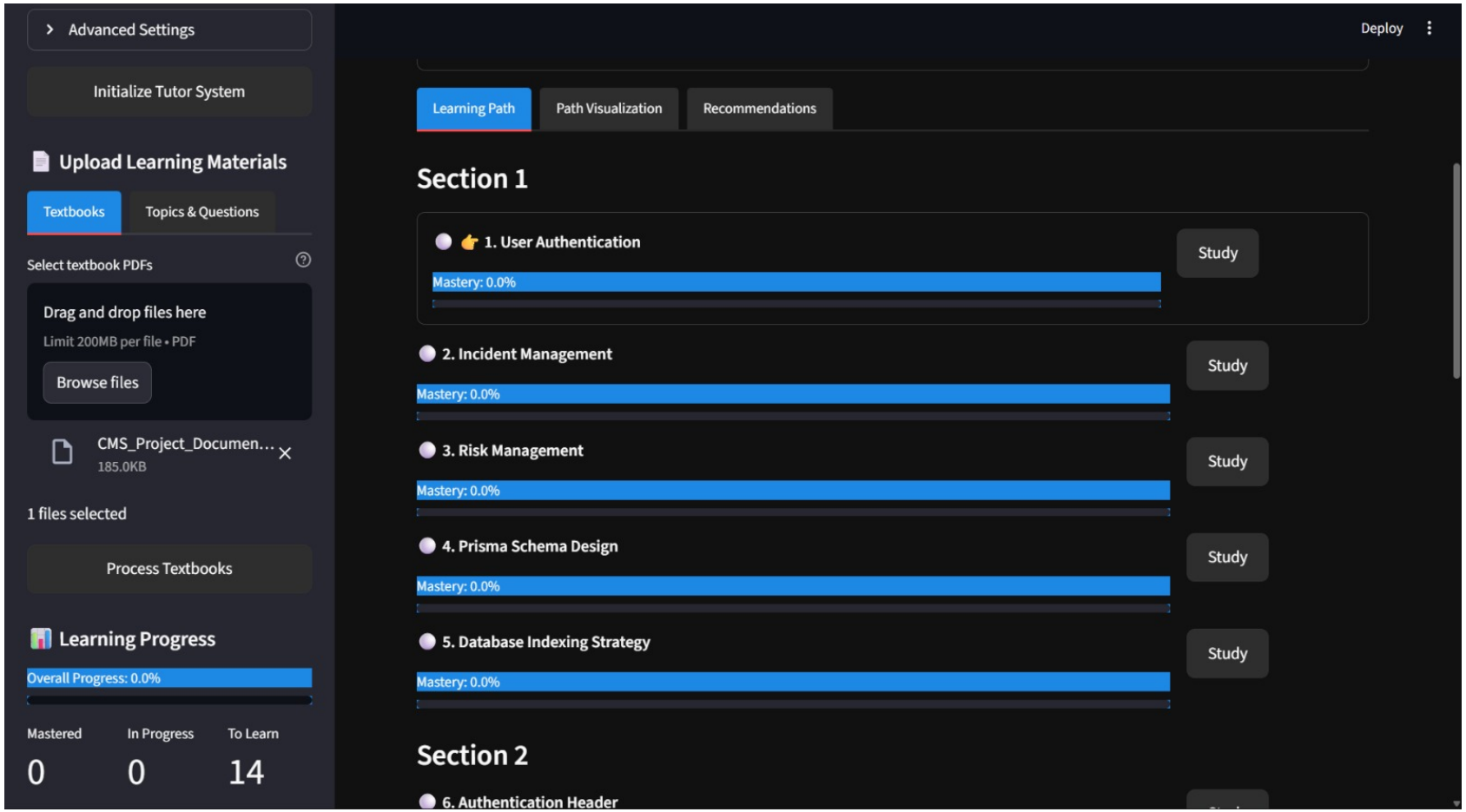

Figure 4.3: Learning Path – Topologically Sorted Study Sequence

The Learning Path displays topics in dependency order across two sections, with each topic showing a mastery progress bar at 0.0% and a dedicated "Study" button. This sequenced view ensures students encounter prerequisite concepts before advancing to more complex material.

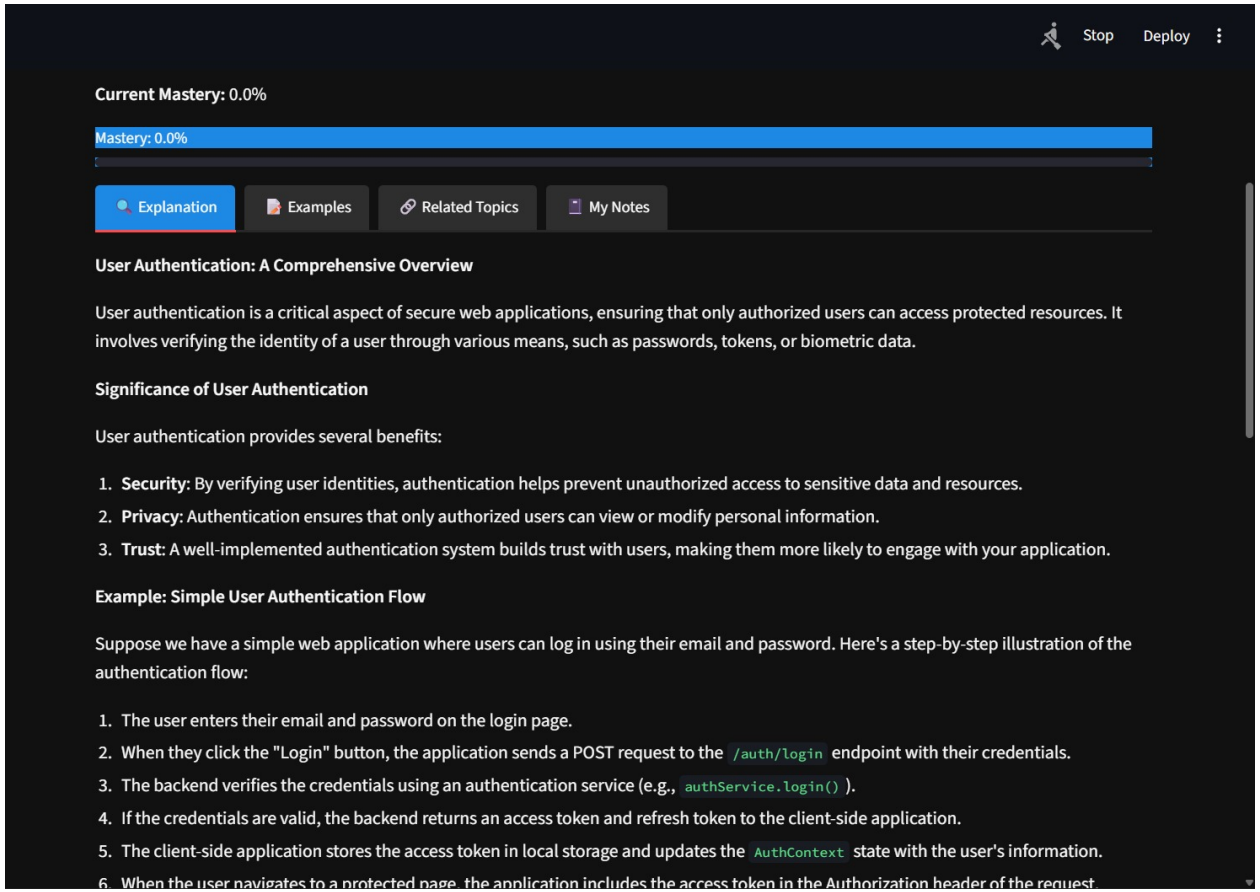

Figure 4.4: Study Mode – AI-Generated Explanation for "User Authentication"

The Study Mode interface presents an LLM-generated explanation for the selected topic, drawn entirely from the uploaded document via RAG retrieval. The content is organized under structured headings and delivered through four tabs: Explanation, Examples, Related Topics, and My Notes.

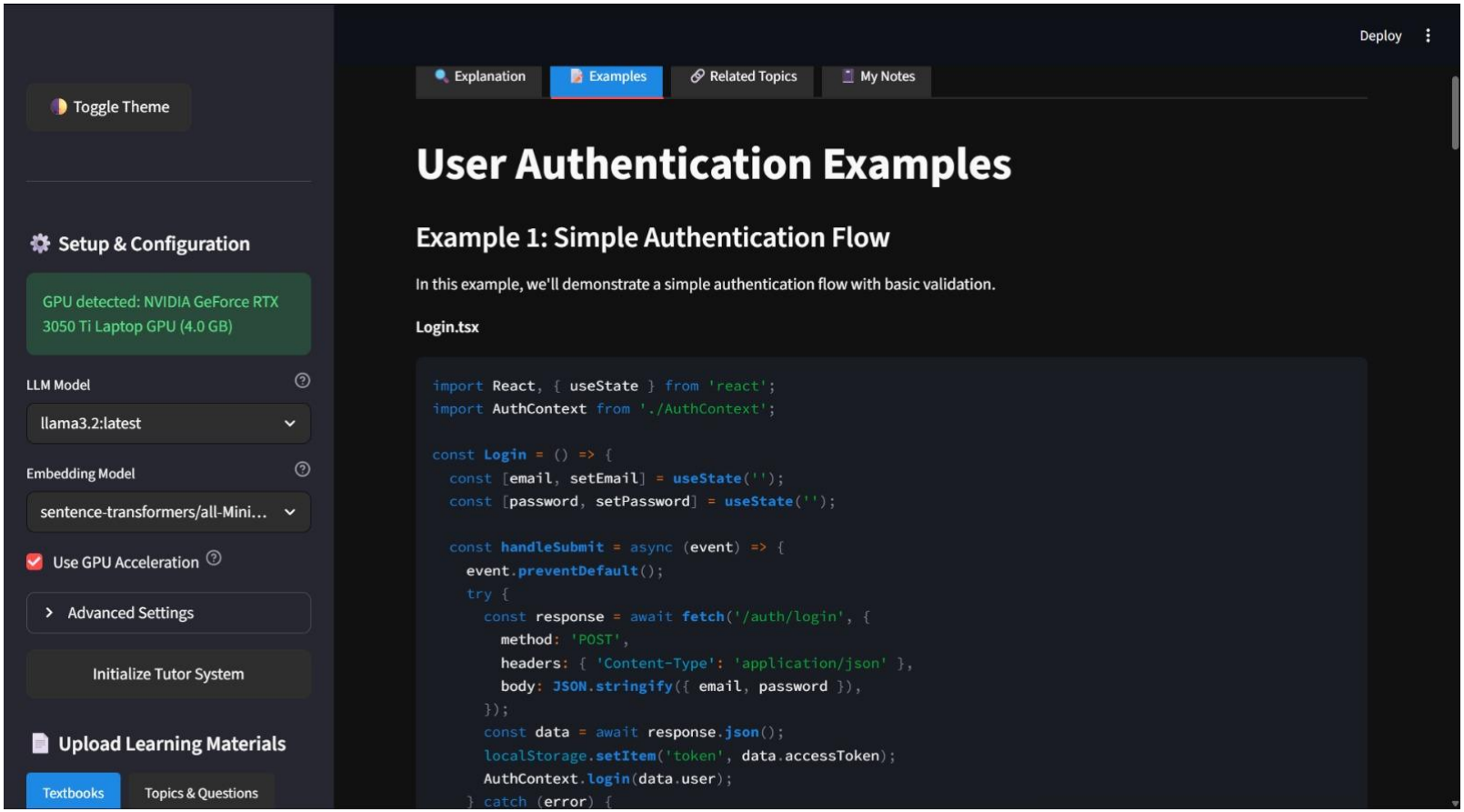

Figure 4.5: Study Mode – Code Example Generated for "User Authentication"

The Examples tab within Study Mode generates a practical, topic-relevant code demonstration — in this case, a Login.tsx React component illustrating a complete authentication flow. This confirms the system's ability to produce technically accurate, document-grounded code examples alongside conceptual explanations.

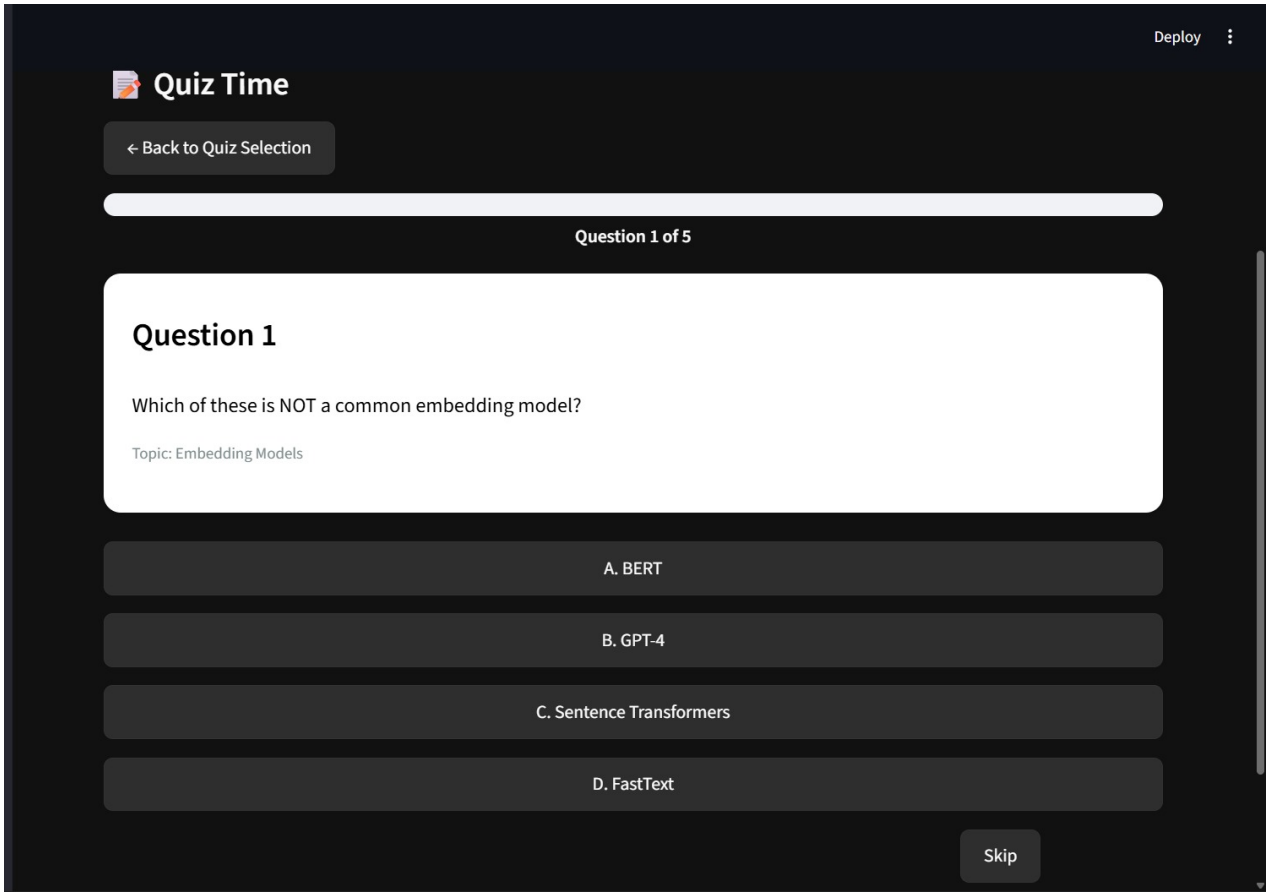

Figure 4.6: Quiz Mode – Adaptive Multiple-Choice Question Interface

The Quiz Mode presents one auto-generated multiple-choice question at a time, with a visible progress indicator (Question 1 of 5) and a Skip option. Questions are drawn from the same retrieved content used during study, ensuring direct alignment between what the student learned and what they are assessed on.

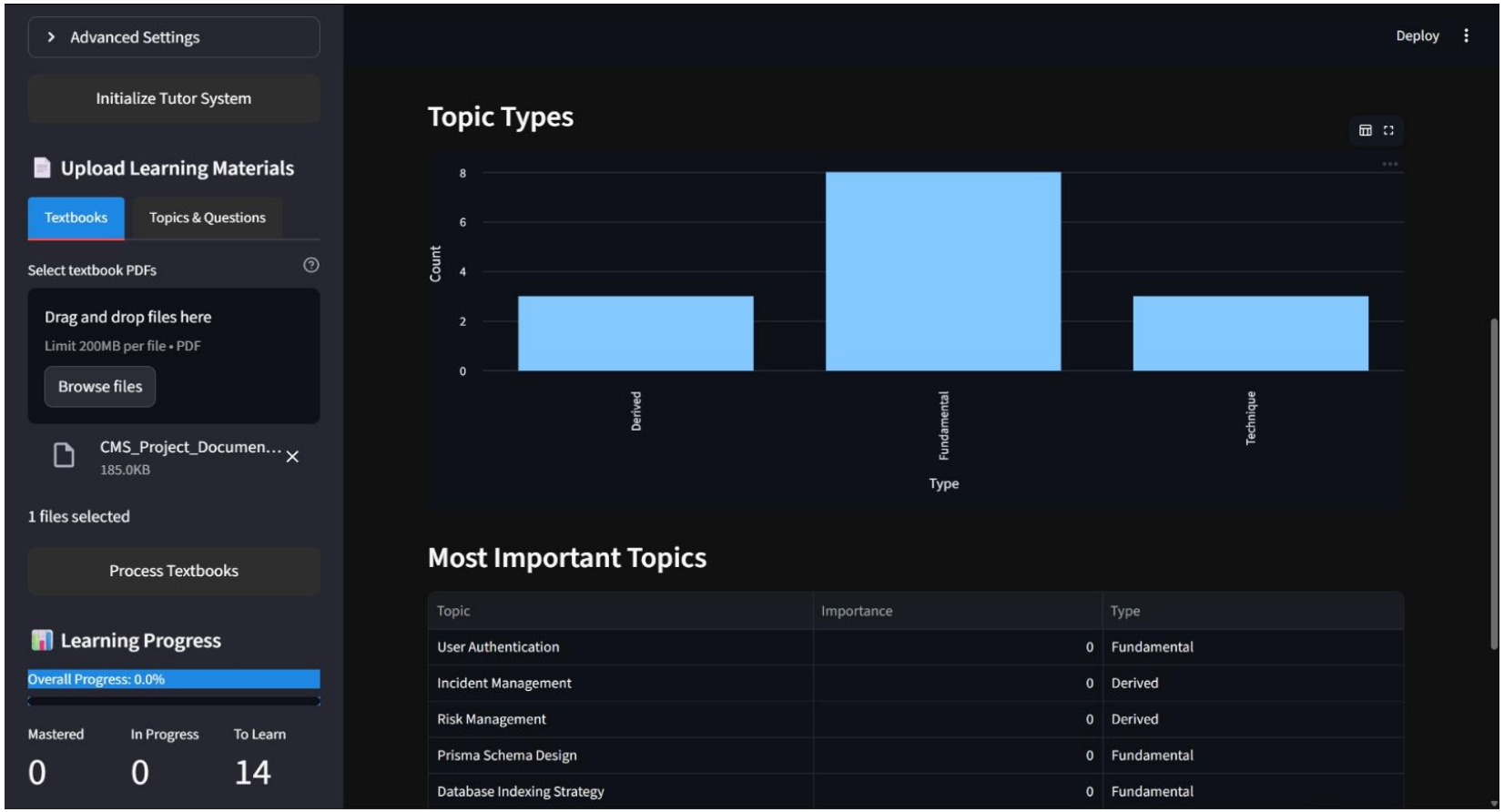

Figure 4.7: Topic Type Distribution Chart

The figure displays a bar chart breaking down the 14 extracted topics by type — Derived, Fundamental, and Technique — giving a quick visual summary of the document's conceptual composition. Below the chart, the Most Important Topics table ranks concepts by their betweenness centrality score, highlighting which topics serve as the most critical connectors in the knowledge structure.

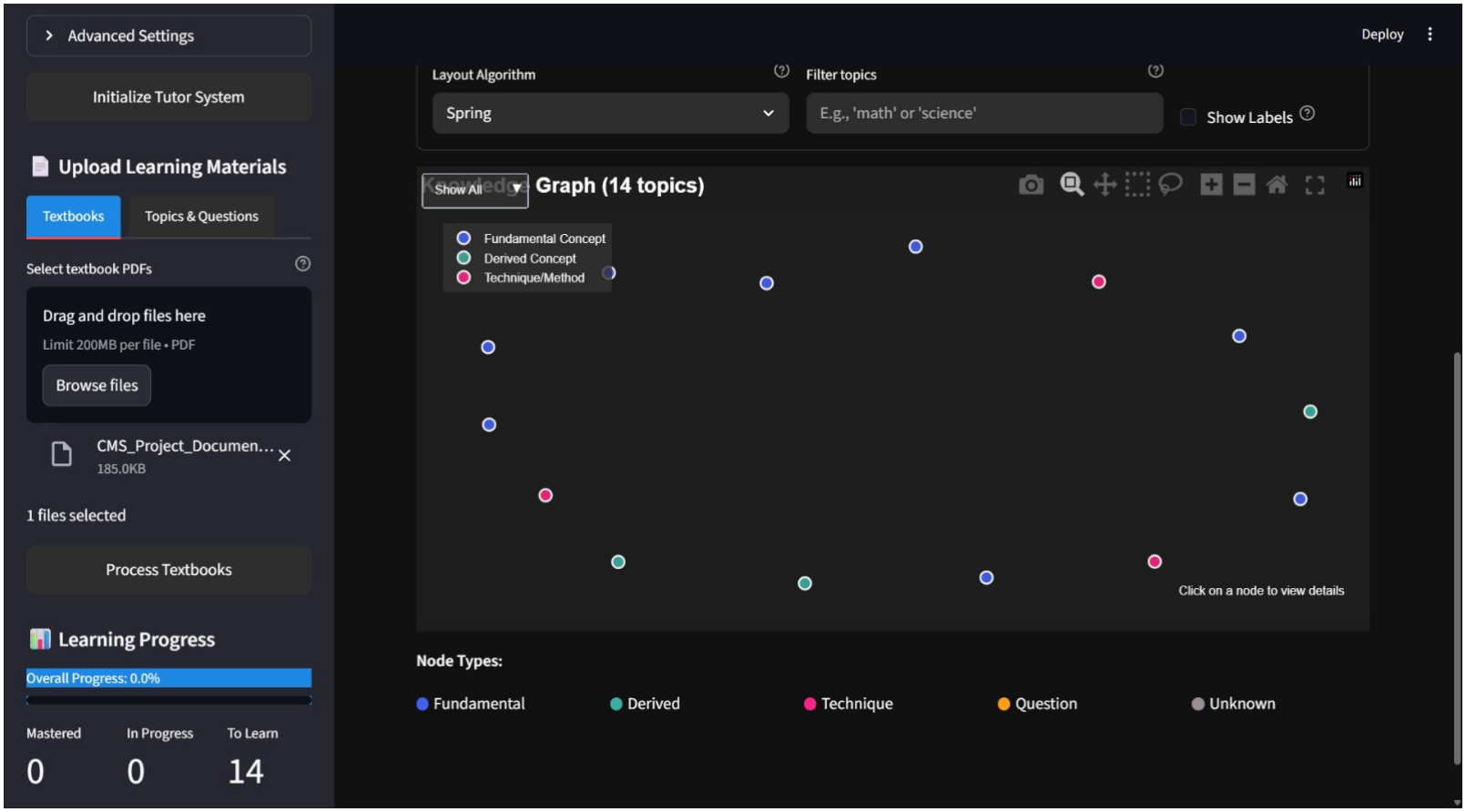

Figure 4.8: Knowledge Graph After Session

A second view of the Knowledge Graph is shown post-session, confirming that the graph structure and node classifications persist correctly throughout the learning workflow. The node type legend and interactive controls remain accessible, allowing students to explore topic relationships at any point during their study session.

4.3 Observations from the work

The evolution of SaraswathAI has been characterized by three successive levels – Mini Project I, Mini Project II, and Major Project. In each level, a specific problem was tackled, building on the strengths of the previous one and adding more technical intricacy. This chapter highlights the major observations made during the course of the Major Project, comparing the three levels of projects in general.

The Evolution at a Glance

Mini Project My mini project aimed at solving one specific problem only; that is, students have difficulty locating the relevant information in different PDF files in response to particular curricular questions. The solution came in the form of a GPU-aided Retrieval-Augmented Generation system that can extract embedded information from a set of uploaded documents, upload the content into the FAISS vector database and use the LLM to respond to students' queries. In total, three methods were utilized to handle the incoming questions; they included Direct Retrieval, Enhanced RAG, and Hybrid mode of operation. Overall, GPU acceleration sped up the process of embedding creation and similarity search by 60%, and the document organization was 35.3% more efficient.

Question asked in Mini Project II was whether we can believe the content that the AI was extracting from. In this project, the platform was upgraded by introducing an Ethereum blockchain layer. The SHA-256 hashing algorithm was used for every document being uploaded in the system, thus creating a unique signature that was stored permanently in the form of a smart contract hosted by the Ganache blockchain node. Before producing any answer, the hash signature of the retrieved content was compared with the blockchain signature to ensure that no manipulation of the source documents had occurred. Every transaction was signed with ECDSA. This yielded a success rate of 98% for all verifications and a traceable audit log for all questions and answers posed on the platform, taking 3-5 extra seconds per each verification and uploading documents at 30% higher speed, as shown in Figure 4.9.

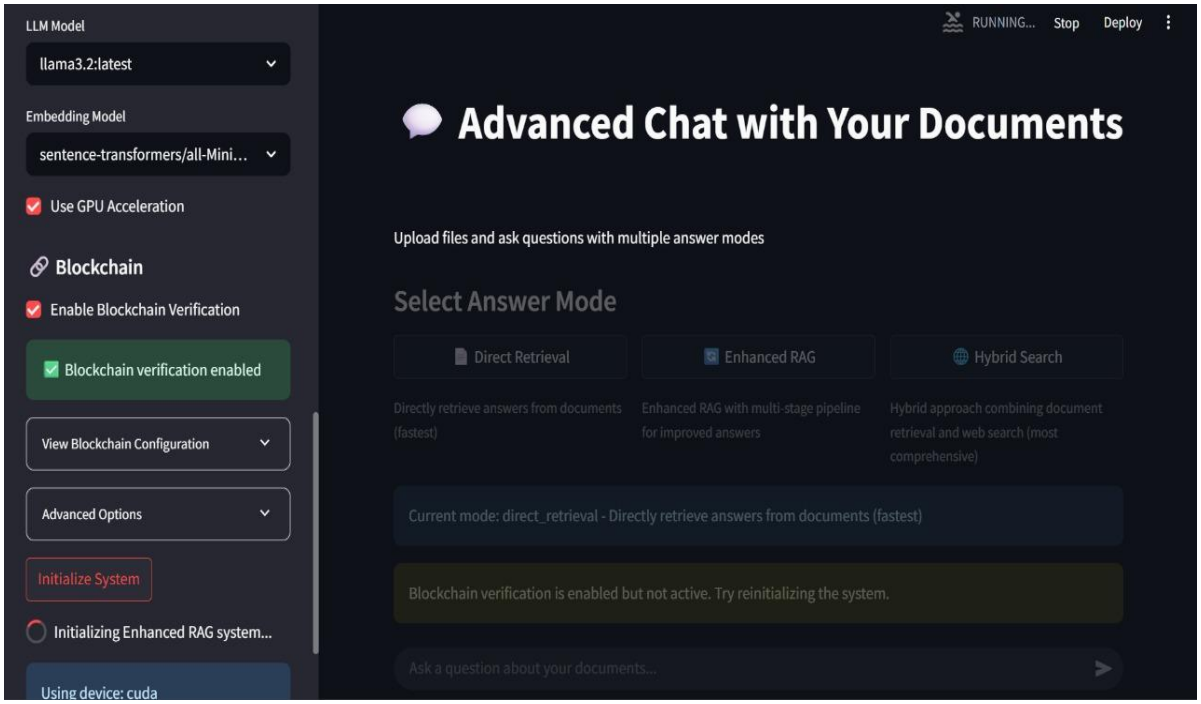

Figure 4.9: Mini Project II – System initialization with blockchain verification activated alongside GPU-accelerated RAG

The Major Project takes a leap that is completely different in nature. While the previous two projects improved the efficiency of content search and verification, the Major Project raises a new question about the next step after the student finds the answer. The previous two Mini Projects, in essence, were advanced retrieval systems – the student asked and received an answer. What makes the Major Project different is that the system takes control of the whole learning process. The knowledge graph is constructed from the uploaded content, a personalized learning plan is created, study mode explains complex concepts in detail, quiz mode tests the progress, while mastery mode monitors the development.

Comparative Observations Across the Three Stages

Functionality and Scope

Mini Project I is an example of a Q&A (query/response) system. The user asked a question, the system provided relevant blocks to compose the answer. The entire process is based only on the actions taken by the user, who uses the system without any information on their background knowledge or next goals.

Mini Project II continued to use this interaction approach but introduced a new component of trust underpinning this process. The learner continues to act as the controller of the entire process, while every answer provided is certified via blockchain technology.

The Major Project completely transforms the interaction between the two. The system becomes proactive by analyzing the file that is uploaded and creating a concept graph. Based on the graph, it establishes the right order of study, provides a description, even without asking a question, poses appropriate questions, based on the student's level of competence, and then updates the live mastery graph after the test session.

Retrieval Architecture

The commonality in all three steps is that their architecture rests on the same RAG backbone: vector search using FAISS, embeddings from sentence transformers, and an LLM that runs locally through Ollama. However, the usage of retrieval varies greatly between steps.

For Mini projects one and two, retrieval was triggered by a student query. FAISS was queried to find chunks that matched semantically the question posed by the student, who then got sent to the LLM for generation.

In the Major Project, retrieval is topic-driven rather than query-driven. When a student selects a topic from their Learning Path, the system retrieves the most relevant chunks for that specific concept and uses them both to generate an explanation and to produce quiz questions. Retrieval is no longer reactive — it is purposeful and structured, serving the broader goal of systematic learning rather than one-off question answering.

Assessment and Feedback

Both Mini Projects I and II lack any capability of assessment. They were just information-retrieving systems, not learning systems. A student would be able to review the answer generated by the system but there was absolutely no way for him to check himself whether he has understood it or not, let alone testing his knowledge regarding the same.

The Major Project offers an entire cycle of assessment. The Quiz Mode will generate the Multiple Choice questions based on the results of information retrieved from each topic. Then the Adaptive Difficulty Algorithm will modify the degree of difficulty of the question depending on how accurately the student answered as well as how quickly he responded. Finally, the Mastery Based Progression Algorithm will update the student's score for the particular topic and set him free from further working on the topic once his score crosses the threshold of mastery.

Knowledge Representation

In Mini Projects I and II, there was no attempt at representing the information in any structured way; the document was viewed simply as an unstructured repository of information. No hierarchy existed in terms of determining importance among topics, the dependencies between various concepts, and overall structure of the discipline in the document.

The Knowledge Graph, however, introduces a knowledge representation methodology. The topic list extracted from the uploaded document is first classified as Fundamental Concepts, Derived Concepts, and Techniques/Methods and then organized into a directed graph, indicating inter-dependencies amongst them. The Betweenness Centrality algorithm picks out the most crucial topics in the graph (topics that link the maximum number of concepts) to help the system identify which topics are vital for the student to focus on. The Topological Sort algorithm uses the graph to create the optimal studying plan.

Performance Metrics

Mini Project I established the baseline performance figures: GPU acceleration delivered a 9.4× speedup in document processing and a 13.2× improvement in similarity search over CPU-only operation. Query response times were under 3 seconds for Direct Retrieval, around 53 seconds for Enhanced RAG, and approximately 86 seconds for Hybrid Mode. The system scaled linearly up to 50,000 documents, as depicted in Figure 4.10.

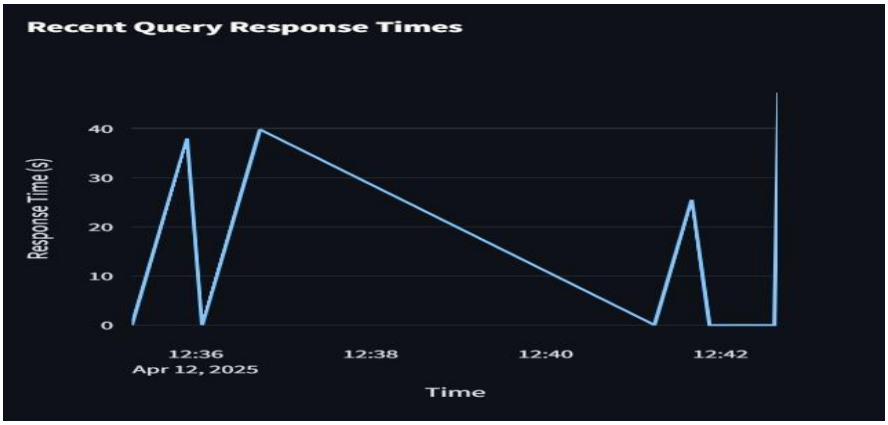

Figure 4.10: Mini Project I – Query response time

While Mini Project II had similar performance features to the RAG pipeline, this time there was additional blockchain latency. Upload times grew to about 30%. Verifying each query took 3-5 extra seconds. Nevertheless, 98% verification and 95% smart contract deployments were successfully completed, taking an average of 3.5 seconds per query verification and 13.4 seconds for smart contracts to be deployed, as can be seen in Figure 4.11.

In contrast, Major Project changes the focus of the performance discussion away from the speed of calculations and to the actual learning process. While the platform is powered by the same GPU-accelerated environment and embedding generation/retrieval is fast, mastery progression becomes the new factor of interest. Mastery progression means how well and how quickly a user goes through "To Learn" and reaches "Mastered" status on various topics within a single document. This progress needs to be done correctly by means of a dynamically adapting quiz system.

Blockchain and Trust

Mini Project II introduced blockchain verification as a core architectural feature. The Major Project deliberately does not extend this layer further — the focus has shifted entirely to the learning experience. However, the architectural foundation laid in Mini Project II remains compatible with the Major Project's pipeline, and blockchain-backed content verification could be reintegrated in a future iteration where institutional accountability requirements demand it.

```
✅ All blockchain tests passed!
Connection test results: {'connection': True, 'network': {'chain_id': 1337, 'gas_price': 200000
 'block_number': 3}, 'account': True, 'balance': Decimal('99.97809064'), 'contract': True, 'rea
': True, 'errors': ["Warning: Contract read confirmation failed: {'message': 'VM Exception whil
essing transaction: revert', 'stack': 'CallError: VM Exception while processing transaction: re
n    at Blockchain.simulateTransaction (C:\\\\Program Files\\\\WindowsApps\\\\GanacheUI_2.7.1.0
rb4352f0jd4m2\\\\app\\\\resources\\\\static\\\\node\\\\node_modules\\\\ganache\\\\dist\\\\node\
s:2:72658)', 'code': -32000, 'name': 'CallError', 'data': '0x'}"], 'simulation_mode': False}
Blockchain verification activated successfully!
```

Figure 4.11: Mini Project II – Blockchain test status report

User Experience

Mini Project I offered a chat-style interface through Streamlit, with a sidebar for document management and mode selection as shown in Figure 4.12. The interaction was conversational and familiar.

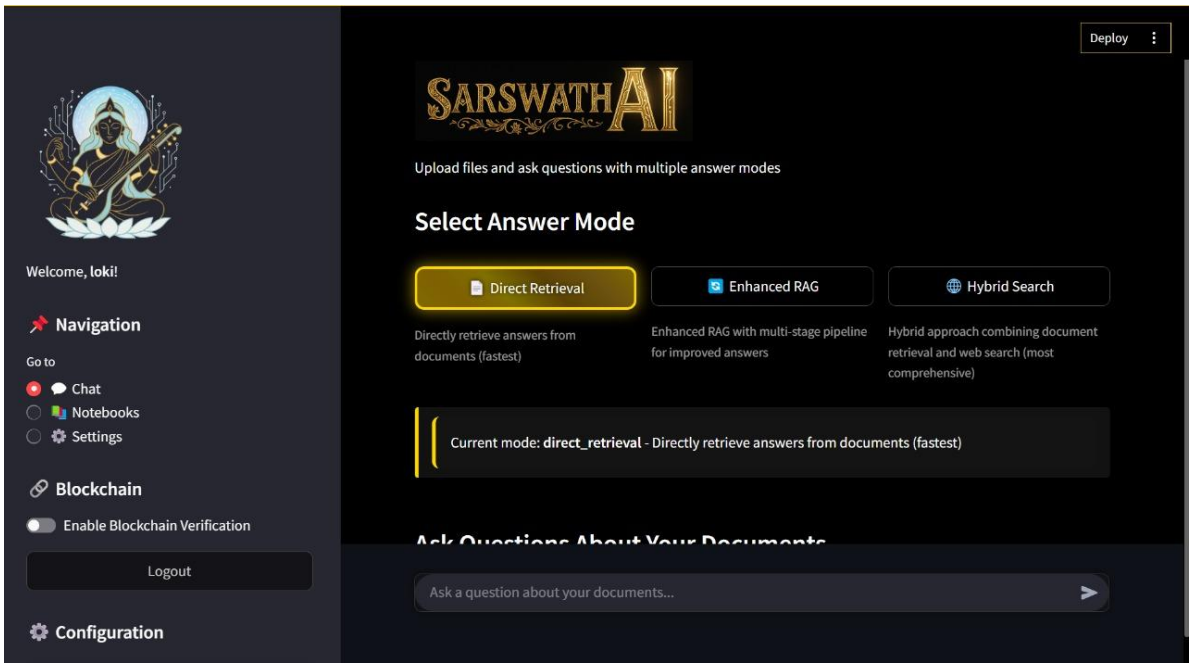

Figure 4.12: Mini Project I – Chat-based home interface

Mini Project II kept everything similar to what was provided, except it had a Verification Status pane and blockchain transactions details pane; however, it did not change the navigation pattern of a student in any major way.

On the other hand, in the Major Project, this experience is completely redesigned. Now, the dashboard is centred around the student's experience in the system, and students are required to navigate through four intentionally different interfaces – Interactive Graph Explorer for the Knowledge Graph mode, Progress Indicator for the Learning Path mode, Explanation Viewer for the Study Mode, and Question Interface for the Quiz Mode. In this case, the side bar continuously displays the progress of the student. This experience seems more like tutoring rather than search-based.

Table 4.1: Summary of Comparative Analysis

Dimension	Mini Project I	Mini Project II	Major Project
Core purpose	Document Q&A	Verified document Q&A	Agentic personalised learning
Interaction model	Student queries → system answers	Student queries → verified answer	System guides → student learns
Knowledge representation	Flat chunks	Flat chunks + blockchain hashes	Knowledge Graph with typed nodes
Study sequencing	None	None	Topological Sort on dependency graph
Assessment	None	None	Adaptive MCQ quiz per topic
Mastery tracking	None	None	Real-time per-topic mastery scores
Blockchain integration	None	SHA-256 + ECDSA + Smart Contracts	Not extended (compatible foundation)
Verification success rate	N/A	98%	N/A
GPU processing speed	0.5–1 ms/page	0.5–1 ms/page (+ 30% upload overhead)	Same baseline, topic-driven retrieval
Query latency	17s / 53s / 86s	+3–5s per query (verification)	Topic-driven; not query-latency bound
User experience	Chat interface	Chat + verification dashboard	Multi-mode learning dashboard

Overall Observations

Regardless of the stage, there was always only one guiding rule: every single piece of information produced by the AI relies on the student's self-generated data, not any third-party content. What did vary between the three projects is the use case and the role of the user in it.

The Mini Project I showed that RAG empowered by the GPU could be used to create a highly efficient learning tool. The Mini Project II demonstrated how blockchain implementation could turn this application into something that could actually be used in practice. Finally, the Major Project is here to prove that the right architecture applied to the already proven technologies will create a system that is truly educational. Instead of answering questions, the system now builds knowledge.

From the standpoint of an unbiased researcher, such a path looks like a logical sequence of works aimed at solving the same goal. Every step brought us closer to the goal by overcoming an actual challenge. The Major Project is more than Mini Projects I and II – it represents a whole different class of applications.

Chapter -5 Conclusion and Future work

5.1 Conclusion

SaraswathAI was initiated with a simple premise; despite being surrounded by more educational resources than ever before, the problem of getting the right information at the right time still persists. Mini Project I focused on creating a GPU-accelerated Retrieval-Augmentation Generation model capable of searching and returning accurate results within academic texts based on specific curriculum-related queries. Mini Project II took this concept one step further by addressing the issue of verification. An Ethereum-based blockchain layer was implemented to cryptographically secure and provide an immutable audit trail for all interactions between users and documents. The Major Project builds upon these two projects while also adding the final piece of the puzzle; can these systems facilitate learning for their users beyond just providing information?

The answer is affirmative, but only if the system actively intervenes in the process. This is precisely what makes the Major Project different from its predecessors and what represents the main contribution of this project. The SaraswathAI has transitioned from a state-of-the-art document retrieval system into a proactive educational platform that handles the entire learning process autonomously.

These technical advancements, too, hold significance on their own. The Knowledge Graph building pipeline, where topics are extracted from the uploaded document, sorted as Fundamental Concepts, Derived Concepts, and Techniques/Methods, and the dependency relationships between them established, offers the system a real sense of structure for the very first time. The previous stages viewed the document as merely a compilation of information chunks to be retrieved, while the Major Project views it as an organized structure of knowledge that can be reasoned through. The Betweenness Centrality algorithm determines the most crucial topics within the entire subject matter, while the Topological Sort algorithm organizes the subjects into a logical order, similar to how a competent instructor would.

The Study Mode shows that RAG retrieval can have purposes other than just answering questions. While in Study Mode, the system, driven by topics instead of a student's query, uses the same pipeline to provide well-structured, coherent explanations that

even include practical examples and references to other relevant concepts – all based on the student's own uploaded document.

The system does not hallucinate general knowledge; it synthesises specifically from what the student has chosen to study.

Whereas Mini Projects I and II did not provide any feedback mechanism to determine the effectiveness of their approach, the Assessment layer provides just that. The Quiz Mode creates multiple-choice questions based on the retrieved information, with difficulty adjusted based on the student's current performance using the Adaptive Difficulty Algorithm. Using the Mastery Based Progression Algorithm, completion is determined only after the learner demonstrates consistent understanding of the material, rather than reading it simply once. In this way, a cycle of personalized explanation, adaptive assessment and mastery-based progression is established.

Taking into consideration the full course of this research, it is clear that SaraswathAI was built in line with a purposeful trajectory. Whereas Mini Project I showed the ability of the AI algorithm to make the learning material easily accessible, Mini Project II showed how blockchain technology could ensure its integrity. The Major Project combines both achievements to create an agent-based architecture that transforms a knowledge delivery system into a personal teacher.

5.2 Future Study

The current implementation establishes a strong and functional foundation, but several meaningful directions remain open for future development. These are not speculative additions — they emerge directly from limitations and observations encountered during the development and testing of the Major Project itself.

Re-integration of Blockchain Verification

This deliberate move by the Major Project is aimed at completely concentrating on the learning experience through agency. This is a short-term decision that would be revisited soon enough. Due to the fact that both pipelines share similar architectural characteristics, there is room for integrating blockchain-based content verification into SaraswathAI once again, without the need for any rearchitecting whatsoever. In the future, each explanation produced by the AI, alongside each question contained within

the quizzes, will have a blockchain ID leading directly back to the cryptographically verified source chunk of content – essentially marrying the best aspects of both the Mini Project II and the Major Project.

Multi-Document and Cross-Corpus Knowledge Graphs

The current process creates Knowledge Graphs on a per-document basis. However, in practical academic environments, students use several books, lectures, journals, and other references at once. Future development may take advantage of that by applying the process of Knowledge Graph creation on the level of all documents used by a student. This way, all common concepts will be identified, all conflicts between the resources will be settled, and a single Knowledge Graph for all resources used will be created. It will significantly strengthen the Learning Path creation process as well.

Longitudinal Learning Analytics

Currently, the system measures mastery throughout a single lesson. The logical next step to take is to maintain student learning status across multiple lessons, thus creating a long-term history of progress in individual learners' knowledge over days, weeks, and semesters. Such information can reveal areas in which a learner regularly experiences difficulty, help estimate the results in future quizzes, and even allow for intelligent introduction of content not encountered for some time, using spaced repetition, which is widely studied in the field of educational research. For schools and other institutions, anonymous statistics for all students enrolled in particular courses can give insights into the effectiveness of the curriculum and areas in which students have problems.

Open-Ended and Subjective Assessment

Currently, the Quiz Mode only offers multiple choice questions, which are ideal for testing knowledge retention and differentiation skills but not so much for evaluating comprehension, critical thinking, or problem solving skills. In future developments, it will be good to consider introducing a short-answer type of quiz and evaluate answers using the LLM. Feedback should be given in more detail than simply stating if the answer is right or wrong but also explaining what makes the answer good or what makes it suboptimal. Code-based questions, which involve writing a program function or algorithm, can also be considered since the platform creates codes in Study Mode.

Real-Time Web Augmentation

At the moment, the platform works only through uploaded documents, thereby ensuring that any material is relevant and under control since it relates to the curriculum. There are times when the student will require that the material contained in the documents be related to what is happening in the world today, either through new research work or other examples. The Hybrid Mode presented during Mini Project I has proven that the use of both the documents and external sources can be done successfully. Going forward, a web augmented version can be turned on depending on the student's preferences.

Deployment Beyond Desktop

However, currently the system works on a local computer with a GPU that supports CUDA, introducing a hardware requirement which may affect accessibility. In future, two main directions for development can be considered: one is related to cloud deployment, when educational organizations will be able to provide SaraswathAI as a service for everyone without the need to install GPU-enabled hardware; another relates to the possibility of running SaraswathAI on less powerful hardware, by training models with smaller LLMs that sacrifice some quality in exchange for greater accessibility. The creation of a mobile application will allow students to use the knowledge graph on the move and do quizzes on the go.

Integration with Existing Learning Management Systems

There are also several Learning Management Systems such as Moodle, Canvas, Blackboard, and other platforms currently employed by universities and colleges for delivering course material to students, receiving assignments from students and grading students' performances. As opposed to functioning as a separate standalone app, the next iteration of SaraswathAI may become seamlessly integrated with the above-listed platforms via Learning Tools Interoperability (LTI). In this regard, SaraswathAI will be able to read and ingest all documents used in courses, make sure that its Knowledge Graph is aligned with an officially-approved syllabus of courses and provide feedback on students' mastery of material directly in the university's grading system.

References

[1] Fang, Jinyuan, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. "A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems." arXiv:2508.07407 (2025).

[2] Author(s) Unknown. "LLM-Based Educational Content Retrieval and Intelligent Query Processing." *Information Processing & Management* (2026). Elsevier.

[3] Author(s) Unknown. "AI-Driven Learning Interaction and Engagement Framework for Digital Education." *Open Access Ed-Tech Journal* (2026). Elsevier.

[4] Author(s) Unknown. "Personalized Learning Pathways and AI-Guided Academic Improvement in Higher Education." *Nurse Education in Practice* (2026). Elsevier.

[5] Chen, Junyi, Zheyong Xie, Tong Xu, Sheng Zhou, Hongzhi Yin, and Enhong Chen. "Agent4EDU: Advancing AI for Education with Agentic Workflows." ResearchGate (2025).

[6] Veschetti, Adele, Richard Bubel, and Reiner Hähnle. "SmartML: Enhancing Security and Reliability in Smart Contract Development." *Blockchain: Research and Applications* (2025): 100362. Elsevier.

[7] Kulkarni, S., et al. (2024). "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models." ACL 2024.f

[8] Zhang, Y., et al. (2024). "A Comprehensive Survey of Retrieval-Augmented Generation: Evolution, Current Landscape and Future Directions." arXiv:2410.12837.

[9] Kulothungan, K., et al. (2025). "Blockchain-based AI Decision Audit Trail Framework for IoT Environments." IEEE Transactions on Industrial Informatics.

[10] European Union. (2024). "Regulation on Artificial Intelligence (AI Act)." Official Journal of the European Union.

[11] Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. NeurIPS.

[12] Paneru, Biplov, Bipul Thapa, and Bishwash Paneru. "Leveraging AI in Ayurvedic Agriculture: A RAG Chatbot for Comprehensive Medicinal Plant Insights Using Hybrid Deep Learning Approaches." *Telematics and Informatics Reports*, vol. 16, 2024, p. 100181. Elsevier.

[13] Z. Yang, E. Khatibi, N. Nagesh, M. Abbasian, I. Azimi, R. Jain, and A. M. Rahmani, "ChatDiet: Empowering personalized nutrition-oriented food recommender chatbots through an LLM-augmented framework," Smart Health, vol. 32, p. 100465, 2024.

[14] Abbasian, Mahyar, et al. "Conversational Health Agents: A Personalized LLM-Powered Agent Framework." *arXiv*, 26 Sept. 2024, arXiv:2310.02374v5 [cs.CL].

[15] Pichai, Kieran. "A Retrieval-Augmented Generation Based Large Language Model Benchmarked On a Novel Dataset." Journal of Student Research, vol. 12, no. 4, 2023.